A Hybrid Deep Learning Strategy for Real-Time Assembly Task Recognition Based on Hand Joint Trajectories

A Project Report submitted in the partial fulfillment of the Requirements for the award of the degree

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Katekeni Hemanthini (22471A0529)

Under the esteemed guidance of

Dr.R.Sathees kumar, M.Tech, Ph.D

Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NARASARAOPETA ENGINEERING COLLEGE: NARASAROPET

(AUTONOMOUS)

Accredited by NAAC with A+ Grade and NBA under Tyre -1 NIRF rank in the band of 201-300 and an ISO 9001:2015 Certified

Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK, Kakinada

KOTAPPAKONDA ROAD, YALAMANDA VILLAGE, NARASARAOPET- 522601

2025-2026

NARASARAOPETA ENGINEERING COLLEGE

(AUTONOMOUS)

DEPARTMENTOF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project that is entitled with the name **"**A Hybrid Deep Learning Strategy for Real-Time Assembly Task Recognition Based on Hand Joint Trajectories**"** is a bonafide work done by the team **Katekeni Hemanthini (22471A0529)** in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of COMPUTER SCIENCE AND ENGINEERING during 2024-2025.

PROJECT GUIDE
Dr. R. Satheeskumar, M. Tech, Ph.D
Professor

PROJECT CO-ORDINATOR
Dr. Sireesha Moturi, B. Tech.,M.Tech., (Ph.D)
Associate Professor

HEAD OFTHE DEPARTMENT
Dr. S. N. Tirumala Rao, M. Tech., Ph.D.
Professor & HOD

EXTERNAL EXAMINER

DECLARATION

We declare that this project work titled "**A Hybrid Deep Learning Strategy for Real-Time Assembly Task Recognition Based on Hand Joint Trajectories**" is composed by ourselves that the work contains here is our own except where explicitly stated otherwise in the text and that this work has not been submitted for any other degree or professional qualification except as specified.

Katekeni Hemanthini (22471A0529)

ACKNOWLEDGEMENT

We wish to express thanks to various personalities who are responsible for the completion of the project. We are extremely thankful to our beloved chairman sri **M. V. Koteswara Rao**, **B.Sc.**, who took keen interest in us in every effort throughout thiscourse. We owe out sincere gratitude to our beloved principal **Dr. S. Venkateswarlu, M. Tech, Ph.D.** for showing his kind attention and valuable guidance throughout the course.

We express our deep-felt gratitude towards **Dr. S. N. Tirumala Rao, M. Tech., Ph.D.,** HOD of the CSE department and to our guide **Dr. R.Satheeskumar, MTech.,Ph.D** , Professor of CSE whose valuable guidance and unstinting encouragement enable us to accomplish our project successfully in time.

We extend our sincere thanks to **Dr. Moturi Sireesha, B. Tech., M. Tech., (Ph.D),** Associate professor & Project coordinator of the project for extending her encouragement. Their profound knowledge and willingness have been a constant source of inspiration for us throughout this project work.

We extend our sincere thanks to all other teaching and non-teaching staff to department for their cooperation and encouragement during our B. Tech degree.

We have no words to acknowledge the warm affection, constant inspiration and encouragement that we receive from our parents.

We affectionately acknowledge the encouragement received from our friends and those who were involved in giving valuable suggestions had clarifying our doubts which had really helped us in successfully completing our project.

By

Katekeni Hemanthini (22471A0529)

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

INSTITUTE VISION AND MISSION

INSTITUTION VISION

To emerge as a Centre of excellence in technical education with a blend of effective student centric teaching learning practices as well as research for the transformation of lives and community.

INSTITUTION MISSION

M1: Provide the best class infra-structure to explore the field of engineering and research.

M2: Build a passionate and a determined team of faculty with student centric teaching, imbibing experiential, innovative skills.

M3: Imbibe lifelong learning skills, entrepreneurial skills, and ethical values in students for addressing societal problems.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE DEPARTMENT

To become a center of excellence in nurturing the quality Computer Science & Engineering professionals embedded with software knowledge, aptitude for research and ethical values to cater to the needs of industry and society.

MISSION OF THE DEPARTMENT

The department of Computer Science and Engineering is committed to

M1: Mould the students to become Software Professionals, Researchers and Entrepreneurs by providing advanced laboratories.

M2: Impart high quality professional training to get expertise in modern software tools and technologies to cater to the real time requirements of the industry.

M3: Inculcate teamwork and lifelong learning among students with a sense of societal and ethical responsibilities.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

Program Specific Outcomes (PSO's)

PSO1: Apply mathematical and scientific skills in numerous areas of Computer Science and Engineering to design and develop software-based systems.

PSO2: Acquaint module knowledge on emerging trends of the modern era i n Computer Science and Engineering

PSO3: Promote novel applications that meet the needs of entrepreneur, environmental and social issues.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

Program Educational Objectives (PEO's)

The graduates of the program are able to:

PEO1: Apply the knowledge of Mathematics, Science and Engineering fundamentals to identify and solve Computer Science and Engineering problems.

PEO2: Use various software tools and technologies to solve problems related to academia, industry, and society.

PEO3: Work with ethical and moral values in multi-disciplinary teams and can communicate effectively among team members with continuous learning.

PEO4: Pursue higher studies and develop their career in the software industry.

.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

Program Outcomes (PO'S)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis:** Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability:** Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

9. **Individual and teamwork**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

Project Course Outcomes (CO'S)

CO421.1: Analyze the System of Examinations and identify the problem.

CO421.2: Identify and classify the requirements.

CO421.3: Review the Related Literature.

CO421.4: Design and Modularize the project.

CO421.5: Construct, Integrate, Test and Implement the Project.

CO421.6: Prepare the project Documentation and present the Report using appropriate method.

Course Outcomes – Program Outcomes mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
C421.1		✓											✓		
C421.2	✓		✓		✓								✓		
C421.3				✓		✓	✓	✓					✓		
C421.4			✓			✓	✓	✓					✓	✓	
C421.5					✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
C421.6									✓	✓	✓		✓	✓	

Course Outcomes – Program Outcome correlation

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
C421.1	2	3											2		
C421.2			2		3								2		
C421.3				2		2	3	3					2		
C421.4			2			1	1	2					3	2	
C421.5					3	3	3	2	3	2	2	1	3	2	1
C421.6									3	2	1		2	3	

Note: The values in the above table represent the level of correlation between CO's and PO's

1. Low level

2. Medium level

3. High level

Project mapping with various courses of Curriculum with Attained PO's:

Name of the course from which principles are applied in this project	Description of the device	Attained PO
C2204.2, C22L3.2	Gathering the requirements and defining the problem, plan to develop a **Water body Segmentation**	PO1, PO3, PO8
CC421.1, C2204.3, C22L3.2	Each and every requirement is critically analyzed, the processmodel is identified and divided into **seven modules**	PO2, PO3, PO8
CC421.2, C2204.2, C22L3.3	Logical design is done by using the unified modelling language which involves individual teamwork	PO3, PO5, PO8, PO9
CC421.3, C2204.3, C22L3.2	Each and every module is tested, integrated, and evaluated in our project	PO1, PO5, PO8
CC421.4, C2204.4, C22L3.2	Documentation is done by all our four members in the form of a group	PO8, PO10
CC421.5, C2204.2, C22L3.3	Each and every phase of the work in group is presented periodically	PO8, PO10, PO11
C2202.2, C2203.3, C1206.3, C3204.3, C4110.2	Implementation is done and the project will be handled by the Hospital Management and in future updates in our project can be done based on traditional diagnostic techniques.	PO4, PO7 , PO8
C32SC4.3	The physical design includes hardware components like processor, RAM, hard disk.	PO5, PO6, PO8

ABSTRACT

In today's Industry 4.0 landscape, real-time monitoring of manual assembly tasks is vital for boosting operator efficiency, ensuring compliance with procedures, and maintaining high product quality. This project introduces an advanced deep learning based hybrid system for recognizing assembly actions in real time, building upon and extending the HATREC framework. The system processes raw video from a physical workstation, leveraging YOLOv8 for precise tool and component detection and Media Pipe Hands to extract 3D hand-joint movement data. To capture temporal patterns, we padded and normalized these skeletal sequences enriched with hand velocity features and fed them into both LSTM and BiLSTM models. A comprehensive preprocessing pipeline including frame extraction, grayscale resizing, hand tracking, masking, and padding ensures reliable performance across variable length tasks. Tested on seven distinct assembly operations, the system achieved a strong 93 recognition accuracy, outperforming conventional single-modality models while remaining lightweight enough for real-time deployment. By integrating object detection and hand-motion analysis, this solution supports intelligent operator feedback and error reduction on the factory floor advancing smart workforce technologies in next-generation manufacturing.

TABLE OF CONTENT

LIST OF FIGURES

1. INTRODUCTION

In the era of Industry 4.0, the manufacturing sector is undergoing a major digital transformation, where intelligent automation and cyber-physical systems are becoming core enablers of productivity and quality enhancement. Real-time operator monitoring has emerged as an essential requirement to ensure process compliance, reduce variability, and maintain product integrity. Despite rapid advancements in industrial robotics, many manual assembly lines still rely heavily on skilled human workers, especially in customized or small-batch production tasks that demand flexibility and precision. Human-centric operations, however, introduce unpredictability in task execution, creating a strong need for robust monitoring and feedback systems that can interpret human actions accurately in real time.

To address these challenges, research in computer vision-based action recognition has gained significant momentum. Early approaches relied on handcrafted motion features, but such techniques lacked adaptability to diverse environments and could not capture fine-grained movement patterns. Modern deep learning has revolutionized assembly task recognition through models capable of extracting both spatial and temporal information from video data. Networks such as CNNs and LSTMs have demonstrated improved performance by identifying gesture dynamics and learning temporal dependencies in hand motion.

However, methods relying solely on video frames or skeleton features face limitations such as illumination variability, occlusions, and the difficulty of differentiating similar actions that involve identical tools or hand postures. Recognizing overlapping gesture patterns remains a major concern in real-world industrial setups. Object-aware recognition strategies have therefore emerged as a promising direction. By combining object detection with hand-joint trajectory analysis, hybrid models introduce contextual knowledge about the tools involved, significantly improving task understanding. Systems like HATREC have demonstrated the benefits of such multimodal integration.

Building upon this foundation, the present work proposes a hybrid deep-learning framework for real-time hand assembly task recognition. The system integrates YOLOv8 for precise detection of tools and components directly from shop-floor video streams, ensuring robust spatial context extraction.Simultaneously, MediaPipe Hands is

employed to track 3D hand-joint landmarks with high accuracy, generating featurerich trajectories that represent operational movements over time. Velocity-enhanced skeletal features and sequence normalization further strengthen the reliability of temporal modeling.

To effectively learn long-term dependencies in motion patterns, the extracted sequences are processed using LSTM and BiLSTM architectures. The bidirectional variant captures both preceding and upcoming motion cues, enabling improved recognition of transitions within complex gestures frequently observed on the assembly line.The proposed system was validated on a dataset comprising seven distinct assembly operations, covering variations in lighting, hand placement, and operator behavior. Results indicate a 93% recognition accuracy, significantly outperforming baseline methods and demonstrating robustness required for industrial deployment.

By delivering accurate and lightweight real-time recognition performance, this research contributes to the development of smart workforce assistance systems capable of early error detection, adaptive feedback, and improved operational safety. The integration of object-level and motion-level intelligence marks a significant step toward fully interconnected, worker-supportive manufacturing environments in the age of Industry 4.0.

Deep learning and computer vision have greatly strengthened the ability to analyze human activity within industrial environments. Earlier approaches primarily relied on handcrafted features extracted from RGB frames, but they failed to provide reliable performance in dynamic conditions such as lighting variation and hand occlusion. Modern action recognition methods leverage automatic feature extraction using neural networks to capture subtle motion details. Particularly, skeleton-based recognition allows systems to track the hand and body posture of operators, providing a more robust and invariant representation of human action. Manual assembly tasks often involve repetitive and structured hand motions that can be effectively modeled using trajectorybased skeleton data. These techniques enable real-time automation support, facilitating intelligent worker supervision.

As industries move toward smart manufacturing, integrating skeleton tracking with video analytics has become increasingly significant for improved action understanding. In recent years, hybrid frameworks that combine object detection with hand-motion recognition have demonstrated superior performance in assembly monitoring tasks. Tools and parts used during operations provide important contextual information that helps differentiate between visually similar gestures. For instance, screwing and partplacement actions may appear identical in hand motion but involve distinct tool interactions. Object-aware learning allows systems to incorporate environmental cues while interpreting operator behavior more accurately.

The HATREC framework is one such example that integrates object detection and skeleton features to analyze assembly procedures effectively. Its success has motivated researchers to extend this concept further to improve recognition quality, especially under real-world industry conditions. As a result, hybrid approaches are now considered essential in next-generation workforce monitoring systems. Despite substantial advancements, several challenges still impede the widespread deployment of action recognition in industrial settings. Fine-grained gestures occurring in close proximity require highly detailed tracking precision, and overlapping gestures increase classification complexity. Variable task duration introduces temporal inconsistency that must be addressed for stable training.

Furthermore, shop-floor environments often include frequent occlusions caused by tools or operator hand placement. Real-time performance also demands lightweight models capable of deploying efficiently on edge devices rather than relying solely on powerful cloud systems. Current approaches may struggle with balancing accuracy and computational feasibility, indicating that improvements are necessary for robust industrial acceptance. To overcome these concerns, this work introduces a hybrid deep learning strategy that improves assembly task recognition by integrating spatial and temporal learning elements. YOLOv8 is utilized for high-precision object detection of assembly tools and components directly from raw shop-floor video input.

This enables accurate environmental context extraction, supporting improved task interpretation. In addition, MediaPipe Hands is employed to identify 3D joint landmark positions of both hands, producing detailed trajectory-based movement information. Combining tool interactions with hand motion dynamics allows the system to correctly differentiate between visually similar industrial tasks.

Such multi-modal approaches form a strong foundation for real-time assembly classification. The extracted hand-joint sequences are enriched using velocity-based temporal features, normalized, and padded to maintain consistent input length across different task durations. For modeling motion behavior over time, the framework applies Long Short-Term Memory (LSTM) and Bidirectional LSTM (BiLSTM) networks. These architectures excel in learning dynamic patterns from sequence data by retaining historical context during classification. BiLSTM further enhances recognition by analyzing forward and backward motion transitions, which is particularly beneficial in distinguishing subtle gesture variations in assembly processes. With carefully configured architecture and dropout-based regularization, the system ensures reliable generalization across unseen operators and environment settings.

The effectiveness of the proposed hybrid system was evaluated on a curated dataset that includes seven distinct manual assembly operations. These tasks represent typical industrial activities such as tool handling, cable fixing, and valve manipulation performed under variable lighting conditions. Performance evaluation metrics confirm that the BiLSTM-based model achieved 93% recognition accuracy, outperforming baseline LSTM models and earlier HATREC implementations. Confusion matrix analysis reveals strong class-wise performance with minimal misclassification, even in cases involving similar-looking gestures. These results demonstrate the robustness of the proposed approach for real-time deployment in practical assembly environments.

2. LITERATURE SURVEY

Y. Cai and Z. Zhang (2024) address action recognition in structured work procedures by combining skeleton and video features. Their study in Applied and Computational Engineering integrates spatial cues from human body poses with appearance and motion cues from raw video, enabling improved discrimination between fine-grained actions in workflow contexts. This dual-modal approach recognizes both posture dynamics and visual context, which is crucial when actions are subtle or involve small manipulations.

The Anonymous (2024) article in Computers and Industrial Engineering extends this by incorporating deep learning for assembly action recognition and progress prediction. Although author details are withheld, the study situates action recognition within a larger process monitoring framework — using learned representations not only to classify actions but also to predict task progression. This reflects a shift from static classification to temporal sequence understanding and predictive modeling, which is vital for proactive decision support in manufacturing.

Human action recognition has been widely studied in the context of speech processing, computer vision, and industrial automation. Early foundational work by Graves and Schmidhuber [3] introduced Bidirectional Long Short-Term Memory (BiLSTM) networks for framewise phoneme classification. Their work demonstrated the importance of bidirectional temporal modeling, which later became a core technique for sequential data such as gesture and action recognition.

Recent studies have focused on skeleton-based and hand-centric action recognition for industrial environments. Wu and Ma [4] proposed a framework that leverages foundation models and automatic data augmentation techniques combined with skeletal keypoints for hand action recognition in industrial assembly lines. Their approach improved robustness to limited data and variations in operator behavior, highlighting the importance of skeletal representations for fine-grained actions.

Attention mechanisms have also been explored to improve temporal and spatial feature learning. Xu et al. [5] introduced a time–space separable attention network for semi-supervised anomaly detection, demonstrating how attention can enhance feature discrimination in

complex temporal sequences. Although focused on anomaly detection, their method is relevant for assembly monitoring where abnormal actions must be detected.

Feature optimization techniques have been explored to enhance system performance. Shan and Ma [6] applied genetic algorithm–based feature selection for intrusion detection systems in large-scale environments. Their work emphasizes the role of intelligent feature selection in improving classification accuracy and reducing computational overhead, which is also applicable to real-time action recognition systems.

Multi-view perception has gained attention for improving robustness in human–robot interaction scenarios. Bandi and Thomas [7] proposed an action recognition framework using multi-view perception and feature tracking, enabling more accurate recognition under occlusions and viewpoint changes. This approach is particularly relevant in industrial assembly lines where camera placement and occlusion are common challenges.

Comprehensive reviews on skeleton-based action recognition have been presented by

Liu et al. [8] and Chen [9]. These studies systematically analyzed traditional and deep learning–based methods, including CNNs, RNNs, LSTMs, GCNs, and attention-based models. They highlighted that skeleton-based methods offer better generalization, lower sensitivity to background noise, and improved privacy compared to RGB-based approaches.

Object detection plays a crucial role in understanding task context. Ultralytics [10] introduced YOLOv8, a real-time object detection model known for its high speed and accuracy. YOLOv8 has been widely adopted in industrial vision systems for detecting tools and components, enabling contextual awareness in action recognition pipelines.

Hand gesture monitoring systems have further evolved with lightweight frameworks. Meng et al. [11] proposed a real-time hand gesture monitoring model using MediaPipe's registerable system, demonstrating reliable 3D hand joint tracking with low computational cost. This makes MediaPipe highly suitable for real-time industrial applications.

Handling data imbalance remains a major challenge in industrial datasets. Popoola et al. [12] addressed this issue using a multi-phase deep learning framework for industrial IoT data, improving classification performance under skewed data distributions. Their findings are relevant for assembly datasets where certain actions occur less frequently.

Hybrid feature fusion approaches have also been explored. Liu et al. [13] proposed a skeleton-based assembly action recognition method that fuses multiple features for human–robot collaborative assembly. Their results showed improved recognition accuracy by combining spatial and temporal skeletal information.

To improve generalization across tasks and environments, Schoonbeek et al. [14] introduced supervised representation learning for generalizable assembly state recognition. Their work emphasized learning robust representations that transfer well across different assembly setups.Although focused on cybersecurity, graph-based modeling techniques introduced by

Cheng et al. [15] demonstrated the effectiveness of graph representations in modeling complex relationships. Similar graph-based ideas have influenced skeleton-based action recognition through Graph Convolutional Networks.

Finally, Rao et al. [16] applied machine learning techniques for fake profile detection, reinforcing the effectiveness of supervised learning models in pattern recognition tasks, which conceptually aligns with action classification problems.

3. SYSTEM ANALYSIS

3.1 Existing System

The existing system analysis plays a significant role in understanding the operational challenges, limitations, and requirements of manual assembly-based human action recognition systems in modern industrial environments. As manufacturing continues transitioning toward Industry 4.0, processes demand more transparency, reliability, safety, and real-time decision support. Existing systems, however, still rely heavily on manual supervision or partial automation, which makes them vulnerable to inconsistencies, procedural deviations, and human fatigue. Manual assembly processes often involve delicate, fine-grained hand motions such as finger articulation, rotational adjustments, tool manipulation, and precise alignment.

Recognizing such actions accurately requires multimodal fusion of spatial, skeletal, and temporal features—capabilities that many legacy systems fail to provide. This section examines how system evolution, technological maturity, and operational needs collectively influence the design of hybrid deep learning frameworks for action recognition. By analyzing the gaps and limitations of earlier technologies, we highlight why integrating YOLOv8-based tool detection, MediaPipe hand-joint trajectories, and LSTM/BiLSTM temporal modeling becomes essential for achieving high performance, robustness, and scalability in real-world industrial settings.

In the current industrial environment, several systems have been developed to recognize human actions and monitor assembly processes, but most of these systems rely on limited forms of data processing. Traditional assembly monitoring solutions were largely manual, and human inspectors were responsible for watching workers and ensuring that assembly steps were followed correctly. These manual systems are slow, inconsistent, and prone to human error. As industries moved toward automation, early computer vision systems attempted to replace human observation, but their performance was restricted by simple algorithms that could not handle the complexity of real-world factory environments.

First, many studies rely only on 2D skeletons, which lose depth information and cannot represent subtle hand rotations or finger movements. Second, these skeletal-only approaches ignore the presence of tools and assembly components. For tasks where tools

determine the correct classification—such as tapping a valve or placing white parts—skeletal data alone is insufficient.

Additionally, most existing systems depend on standard sequential models like LSTMs that analyze data only in one direction. They do not consider future context, which is essential in fine-grained industrial tasks where the meaning of a hand movement becomes clear only when the entire motion is completed. Furthermore, many existing systems do not include strong preprocessing pipelines to handle missing frames, inconsistent video lengths, or incomplete skeletal tracking. As a result, models trained on noisy or incomplete data often fail to generalize well when tested in real industrial environments. Because of these shortcomings, existing action recognition systems are not reliable enough for real-time operator monitoring or for providing corrective feedback on assembly lines.

Over time, deep learning approaches—primarily involving convolutional neural networks (CNNs)—became more popular in action recognition. CNN-based video models improved the extraction of spatial features and were able to analyze multiple frames to recognize actions. However, these CNN-only systems still did not perform well in industrial assembly tasks where fine finger movements, object manipulation, and tool usage were critical. The CNN frameworks treated the entire video frame uniformly, without paying special attention to the hands, tools, or small components involved in assembly processes. As a result, they failed to distinguish between visually similar tasks like screwing two different parts or placing components of similar shape.

Another significant category of existing systems utilized skeletal tracking technologies, such as those implemented using OpenPose or Kinect sensors. These systems improved the ability to track human joints and body motion. They were useful in recognizing broad, full-body activities such as sitting, standing, jumping, or lifting. However, when applied to industrial assembly tasks—which require detailed interpretation of hand and finger movements—these skeletal-only systems fell short. The majority of them extracted only 2D joint coordinates without depth information, which is crucial for understanding hand rotations or distinguishing between hand movements that appear similar in two dimensions but differ significantly in threedimensional space.

Furthermore, skeletal-only systems do not consider the presence of tools or workpieces. For instance, tightening a screw and rotating a small valve may involve similar hand motion trajectories but require different tools. Without object detection integrated into the pipeline, these models lack the contextual information needed for accurate classification. This limitation becomes especially problematic in industrial environments where many tasks share overlapping motion patterns but differ in tool usage, object placement, or the type of component being assembled.

Many existing systems also rely on simple long short-term memory (LSTM) networks to model temporal information. While LSTMs are useful for sequence learning, they process data only in one direction—either past to future—making them incapable of understanding bidirectional motion context. In assembly tasks, a movement can only be correctly identified when considering both the beginning and end of the motion. For example, placing a component and picking up a component may have similar initial motions but differ toward the end. With unidirectional LSTMs, these subtle differences are often lost, leading to misclassification and unreliable performance.

Another limitation of existing assembly recognition systems is the lack of robust preprocessing techniques. Industrial videos often contain inconsistent frame counts, missing skeletal points, blurred frames, and variations in task duration. Many existing models feed such raw data directly into the classifier without normalization or padding. This creates inconsistencies that lead to unstable training and a high rate of errors. Without structured preprocessing steps—such as resizing, grayscale conversion, frame alignment, and feature normalization—the models cannot generalize well when tested with different operators or under varying workplace conditions.

Moreover, many systems from earlier research were developed and tested in controlled lab environments with clean backgrounds, ideal lighting, and minimal occlusions. While these systems reported high accuracy in experimental conditions, their performance dropped sharply when exposed to real-world factory scenarios. Real industrial scenes include unavoidable disturbances such as shadows, background tools, moving hands from nearby workers, and dynamic lighting — factors that existing systems are not equipped to handle. This difference between lab conditions and realistic industrial environments creates a significant gap in the practical usability of these existing systems.

In addition, existing solutions often suffer from high computational requirements. Some deep learning models rely on heavy architectures that require powerful GPUs and large memory resources. This makes them unsuitable for deployment on edge devices or low-power industrial hardware. Real-time performance is essential in assembly monitoring, yet many existing models process frames too slowly or require complex multi-stage pipelines that introduce delays. These limitations prevent their adoption in factories where speed, reliability, and efficiency are crucial.

Lastly, existing systems typically lack integration between tool detection, hand tracking, and temporal understanding. They operate in siloed modes—either focusing on tools, hands, or full-body actions. Without combining these inputs, existing systems fail to produce a holistic understanding of the assembly process. This fragmented approach causes frequent errors, especially in tasks requiring precise coordination between hand movements and tools. As a result, current systems remain inadequate for the demands of Industry 4.0, where intelligent, real-time monitoring and accurate action recognition are essential.

3.2 Research Gaps

Although several studies have explored deep learning and computer-vision techniques for monitoring industrial assembly tasks, significant research gaps still remain, limiting the accuracy, robustness, and real-time applicability of existing systems. One major gap is the dependence on single-modality information, where earlier models typically rely either on video features alone or skeletal data alone. Videoonly models struggle with fine hand movements and tool interactions, while skeletononly models ignore the presence of objects and components, making them ineffective for identifying tasks that involve similar motions but different tools. This lack of multimodal integration reduces the system's ability to accurately classify complex or overlapping assembly gestures.

Another gap arises from the limited ability of existing systems to handle noisy or inconsistent input data collected from real-world factory environments. Industrial video streams often contain missing frames, partially detected hand joints, occlusions, and varying lighting conditions. Many earlier frameworks do not implement strong

preprocessing techniques such as padding, normalization, or missing-value handling. As a result, the models trained on such inconsistent data frequently exhibit low generalization and unstable performance when tested with different operators or environmental settings.

A further research gap is related to insufficient temporal modeling of hand-joint movements. While some studies employ LSTMs to capture motion sequences, they typically use unidirectional models that consider only past information. Industrial assembly tasks often include subtle transitions and repetitive gestures where the complete motion sequence—from beginning to end—is essential for accurate recognition. Without bidirectional analysis, systems fail to differentiate between tasks that share similar intermediate movements but differ in their final actions. This limitation leads to confusion between tasks such as multiple types of screwing or placing operations.

Moreover, existing research often focuses on models that are computationally heavy and not optimized for real-time deployment. Techniques like CNN-LSTM hybrids, 3D CNNs, and deep multi-camera systems require substantial processing power, causing delays in inference. In practical assembly lines, real-time response is critical for operator support, quality assurance, and error detection. The absence of lightweight yet accurate models is a major limitation in bringing research solutions into actual industry environments.

Another important research gap concerns the limited discrimination of finegrained hand gestures, which are extremely common in manual assembly tasks. Existing methods often perform well when gestures are broad and easily distinguishable, such as lifting or placing large objects. However, industrial assembly involves micro- movements of the fingers, such as rotating a screw, fitting a tiny component, or aligning two parts. Earlier models fail to capture these subtle actions because they either lack high-resolution hand tracking or do not incorporate depth-based joint coordinates. Without accurate modeling of finger-level movements, previous systems frequently misclassify tasks that appear nearly identical, reducing the reliability of task-level recognition.

There is also a notable research gap in handling task variability among different operators. In real manufacturing units, no two workers perform a task in exactly the

same way. Differences in hand speed, assembly style, finger positioning, and tool handling introduce natural variations in the dataset. Many previous studies used limited or homogeneous datasets where motion patterns were consistent across participants. As a result, the developed models were not exposed to diverse working styles. This lack of diversity reduces generalization, causing poor model performance when deployed in real settings where operators vary significantly in their execution patterns.

A further gap is related to the lack of robust feature-engineering approaches that incorporate motion dynamics. Earlier frameworks largely rely on static joint positions or raw video frames without integrating velocity, acceleration, or movement flow patterns. However, hand velocity plays a crucial role in distinguishing assembly stages—for example, rapid rotation during screwing differs from slow, controlled placement actions. Without incorporating these temporal dynamics into feature extraction, earlier systems fail to capture meaningful trends that could significantly improve classification accuracy. This gap limits the richness of the temporal information available for deep learning models.

There is also a gap in integration between task recognition and real-time operational feedback, which is essential for Industry 4.0 environments. While many studies focus solely on classifying gestures, they do not compare system-generated task timing with human-annotated task timing, nor do they monitor task flow in real time. Because of this gap, previous methods cannot be used for automatic progress tracking, early error detection, or operator guidance on modern assembly lines. Without real-time temporal alignment, earlier systems remain research prototypes rather than deployable industrial solutions.

Another significant research gap exists in the area of computational efficiency and hardware adaptability. Many models in earlier literature rely on heavy CNN architectures or complex multi-camera setups that demand powerful GPUs and highend hardware. Industrial environments, however, require lightweight, energy-efficient systems that can run on embedded devices or edge processors. Due to this mismatch, most previous models are impractical for real-time shop-floor deployment. There is a growing need for hybrid approaches that deliver high accuracy while remaining computationally efficient.

Lastly, there is a major gap in the fusion of object-context information with handjoint trajectories. Although some models use object detection and others use skeletal data, very few systems effectively combine both streams at the feature level. Object context— such as which tool or component is being manipulated—is essential for differentiating between similar tasks. Without this fusion, earlier systems overlook critical contextual cues and rely solely on motion patterns. This leads to confusion between tasks involving similar motions but different objects, which is a frequent scenario in manufacturing processes. The absence of strong multimodal fusion techniques remains one of the most notable shortcomings in existing research.

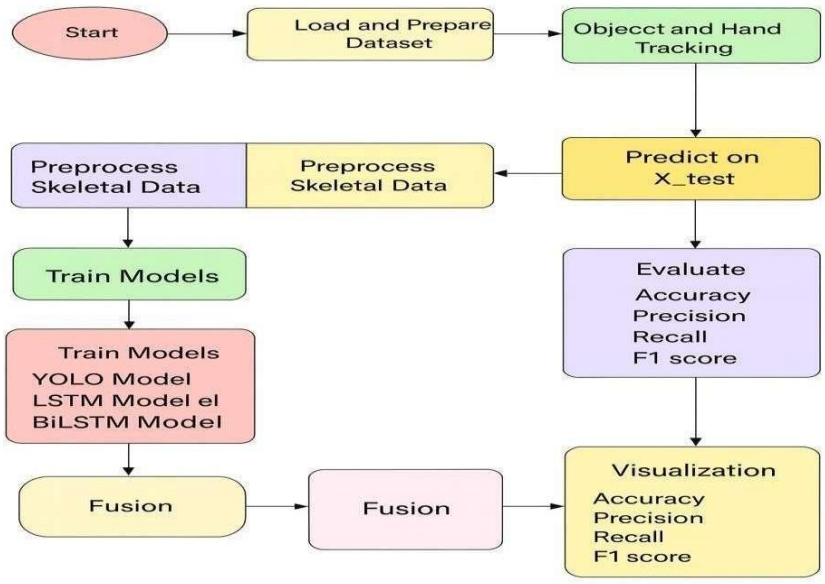


Fig.3.1. Flow chart of Proposed System

3.3 Proposed system:

· Advanced preprocessing including outlier removal, categorical encoding, and normalization.

· Feature engineering with domain-specific composites such as price-per-kilometer and horsepower-to-age ratio.

· Stacked ensemble model leveraging linear and non-linear regression learners for improved accuracy.

· Flask-based deployment with API integration for real-time accessibility across dealerships, fintech, and online platforms.

The proposed system achieves high reliability on the Kaggle Indian used car dataset, with an R^2 of 0.9886, RMSE of ₹1.79 Lakhs, and MAE of ₹0.90 Lakhs, outperforming existing single-model approaches.

Well-defined objectives for the project are:

Develop an Ensemble-Based Prediction Model:

Design and implement a hybrid stacking framework by combining Ridge Regression, XGBoost, and LightGBM. This layered approach captures both linear dependencies and non-linear interactions within the dataset, ensuring stronger predictive performance compared to single-model techniques. The stacking architecture leverages the complementary strengths of base learners, resulting in improved accuracy, reduced variance, and enhanced generalization.

Adapt to Indian Market Dynamics:

Tailor the solution to the unique characteristics of the Indian used car market byincorporating heterogeneous data such as brand reputation, vehicle mileage, age, transmission type, and fuel category. Regional pricing variations and diverse ownership trends are also integrated into the system, ensuring that the predictions reflect real-world conditions. By doing so, the model bridges the gap left by existing global studies that fail to capture local market nuances.

Enable Real-Time Deployment:

Develop an API-enabled solution using a Flask-based framework that allows seamless integration with digital platforms including online marketplaces, dealer networks, and fintech applications. This real-time prediction capability empowers users to obtain instant and reliable valuations by simply providing vehicle details. The deployment ensures scalability, allowing the system to handle large volumes of concurrent requests efficiently.

Ensure Accuracy and Interpretability:

Employ robust preprocessing steps, advanced hyperparameter optimization through cross-validation, and error minimization strategies to achieve high predictive accuracy.

In addition, conduct feature importance analysis to highlight the influence of attributes such as brand, mileage, and horsepower on resale value. This interpretability not only increases user trust but also aids stakeholders—such as buyers, sellers, and financial institutions—in making data-driven decisions with confidence.

3.4 Feasibility Study

Technical Feasibility:

The proposed hybrid deep-learning system—combining YOLOv8 for object detection, MediaPipe Hands for 3D skeletal tracking, and BiLSTM for temporal action recognition—is technically feasible because all required technologies are stable, widely supported, and compatible with modern hardware. YOLOv8 provides high-speed detection with GPU acceleration and lightweight inference, making it suitable for realtime industrial environments. MediaPipe Hands is optimized for efficient hand-joint tracking, offering accurate landmark extraction even in moderately challenging conditions. The BiLSTM model, used for learning motion sequences, requires moderate computational resources, and training can be accomplished using commonly available GPUs. The preprocessing pipeline (frame extraction, padding, normalization) further enhances overall system stability. Therefore, the system can be implemented with existing computer vision frameworks and open-source libraries, making the technical requirements reasonable and achievable.

Hardware Feasibility:

This evaluates whether the required hardware resources are available and sufficient for system implementation.The proposed system needs only standard industrial cameras for video capture and a mid-range GPU (such as NVIDIA GTX/RTX series) to support real-time object detection and model inference. No specialized sensors like depth cameras or Kinect devices are required. The system's use of lightweight algorithms such as YOLOv8 and MediaPipe Hands ensures that it can run efficiently on commonly available machines. Therefore, the hardware requirements are minimal and easily supported.

Software Feasibility:

This determines whether the necessary software tools and frameworks are available, compatible, and stable. The system uses widely adopted and open-source technologies including Python, OpenCV, YOLOv8, MediaPipe Hands, and deep-learning libraries like

TensorFlow or PyTorch. These frameworks are well-supported, regularly updated, and easy to integrate into industrial applications. The preprocessing steps like frame extraction, resizing, and normalization can be implemented using simple software modules. The compatibility and availability of these tools make the system highly feasible from a software perspective.

Algorithmic Feasibility:

This type assesses whether the algorithms used are capable of solving the problem effectively.The system employs a hybrid deep-learning architecture combining YOLOv8 for spatial tool/object detection, MediaPipe Hands for 3D skeletal feature extraction, and BiLSTM for temporal action recognition. Each of these algorithms has been proven effective in previous research. YOLOv8 offers fast, accurate object detection suitable for real-time use. MediaPipe Hands provides precise hand-joint landmarks essential for finegrained gesture understanding. BiLSTM enhances temporal learning by analyzing sequences in both forward and backward directions. Together, these algorithms make accurate task recognition feasible.

Reliability Feasibility:

This determines whether the system can operate dependably in real conditions. The model's accuracy of ~93% and consistent performance during real-time testing indicate high reliability. The preprocessing pipeline handles missing frames, noise, and sequence variations effectively. YOLOv8 is robust against lighting issues, while MediaPipe Hands maintains stable tracking. These factors ensure reliable operation on factory floors with minimal technical failures.

Operational Feasibility:

From an operational standpoint, the system is designed to function seamlessly within real-time assembly environments. It supports live video input from factory-mounted cameras and processes tasks instantly without interrupting workflow. Operators do not need training because the system runs passively in the background while monitoring assembly actions. The system accurately identifies task progress, tool usage, and hand motions, allowing supervisors to monitor worker performance effectively. The real-time behavior of the model—demonstrated by its ability to closely match human-recorded task durations—ensures that it can be reliably integrated into existing industrial operations. Furthermore, since the system reduces manual monitoring, improves accuracy, and minimizes human error, it is highly feasible from an operational perspective.

Workflow Feasibility:

This evaluates the system's ability to fit into existing workflows.The system operates passively without interrupting workers. No changes in worker behavior or task execution are required. It easily adapts to current industrial processes, making workflow integration smooth and feasible.

User Feasibility:

This type focuses on ease of use for workers and supervisors.Operators do not need training, as the system automatically recognizes tasks and movements. Supervisors get clear, real-time insights for monitoring. The minimal user involvement ensures high acceptance and convenience.

Real-Time Feasibility:

This checks whether the system can function instantly without delays.With fast object detection, efficient skeletal tracking, and optimized BiLSTM processing, the system delivers real-time output. It matches human-recorded task durations and supports live monitoring without lag. Hence, it is operationally feasible for continuous use.

Economic Feasibility:

Economically, the proposed system is cost-effective compared to traditional machine vision setups that require expensive multi-camera systems, depth sensors, or custom-built hardware. The use of open-source models like YOLOv8 and MediaPipe reduces licensing costs significantly. Most mid-range factory hardware—including standard industrial cameras and mid-level GPUs—can support the system without additional expensive investments. Once deployed, the system reduces long-term costs by lowering the need for manual supervision, minimizing assembly errors, and improving overall productivity. Although there is an initial cost related to model training, integration, and testing, the return on investment is high due to increased efficiency and reduced operational losses.

Initial Cost Feasibility :

This considers the cost of setting up the system.The proposed system requires only basic cameras and a mid-range GPU for real-time processing. All major software components—YOLOv8, MediaPipe Hands, Python, and deep-learning frameworks—are open-source, meaning there is no licensing cost. Initial development expenses remain low compared to traditional multi-sensor industrial systems.

Operational Cost Feasibility:

This type assesses the day-to-day expenses of running the system.Since the system uses lightweight algorithms and existing hardware infrastructure, operational costs are minimal. There is no need for specialized maintenance or frequent hardware replacements. Electricity and compute usage remain low due to optimized models. Thus, ongoing operational cost is affordable.

Scalability Cost Feasibility:

This analyzes whether expansion is affordable.The system can be scaled to multiple workstations simply by adding cameras or upgrading GPUs. No redesign or costly restructuring is needed. Scaling up is financially reasonable, making this system suitable for small, medium, and large factories.

Behavioral Feasibility:

The system does not alter the natural workflow of workers, making it behaviorally feasible. Workers continue performing tasks as usual, without needing to change their hand movements or assembly speed. The system passively observes and analyzes their actions. Supervisors and managers benefit from clear, real-time insights without requiring additional training. Since the system focuses on improving quality and safety rather than evaluating workers personally, it is likely to gain acceptance among employees. This smooth adoption ensures that the system aligns well with organizational behavior and workplace culture.

Legal and Security Feasibility:

From a legal standpoint, the system does not store biometric data such as facial features; instead, it captures only hand-joint landmarks and object positions. This reduces legal risk related to worker privacy. Video streams can be processed locally without transmitting sensitive data to external servers, ensuring compliance with organizational security policies. With proper access controls and storage safeguards, the system remains secure and legally compliant for industrial deployment.

4. SYSTEM REQUIREMENTS

System requirements define the hardware and software specifications necessary for the successful implementation and execution of the car price prediction system. These requirements ensure that the system can handle large datasets, process multimodal data (structured tabular features, images, and textual descriptions), and efficiently train machine learning and deep learning models. The following sections outline the hardware and software prerequisites needed for optimal performance.

4.1 Hardware Requirements:

The hardware requirements specify the physical computing resources needed to execute the machine learning and deep learning–based car price prediction models. The system processes large-scale automotive datasets with multiple features such as mileage, brand, age, and also visual car images, requiring sufficient computational power to handle feature engineering and model training effectively. The minimum and recommended hardware specifications are listed below:

- System Type : Intel® Core™ i7-7500U CPU @ 2.40GHz
- Cache Memory : 4MB (Megabyte)
- RAM : 16GB (Gigabyte)
- Storage : 512 GB
- Hard Disk : 4GB

4.2 Software Requirements

Software requirements include all necessary tools, libraries, and frameworks required to develop, train, and deploy the car price prediction system. Since this project uses Python-based machine learning and deep learning frameworks, installing compatible libraries is essential for functionality.

- Operating System : Windows 11, 64-bit Operating System
- Coding Language : Python
- Python Distribution : Anaconda, Flask, Google Colab
- Browser : Any Latest Browser like Chrome
- Tool : Visual Studio Code
- Graphics card : Intel(R) Iris(R) Xe Graphics

The hardware and software requirements listed below ensure that the car price prediction system functions efficiently. The system requires sufficient computational resources to process large volumes of car-related data and to train deep learning and machine learning models effectively. Ensuring the correct installation of software dependencies and frameworks enhances model accuracy, improves training and testing speed, and enables smooth price prediction. Additionally, optimized hardware and software configurations reduce processing latency, support scalable deployment, and make the system reliable for real-time car price estimation across large datasets.

4.3 Requirement Analysis

Requirement analysis is a crucial phase in the development of the machine learning–based car price prediction system. This phase ensures that all the functional and nonfunctional requirements are clearly defined and understood, providing a strong foundation for the design and implementation of the system. Functional requirements focus on the main operations of the system, including data preprocessing steps such as handling missing values, normalization, categorical encoding, and outlier detection. In addition, feature engineering and selection are performed on key attributes such as mileage, age, brand, fuel type, ownership, and repair history to enhance predictive performance. The system then trains multiple machine learning algorithms, including linear regression, decision trees, random forest, gradient boosting, XGBoost, and LightGBM, and evaluates their performance using metrics such as R^2, RMSE, MAE, and MAPE. Finally, the system provides a user-friendly interface that allows buyers, sellers, and dealerships to input details and receive accurate car price predictions.

Non-functional requirements define the system's quality attributes, such as performance, scalability, usability, maintainability, and compatibility. The system should predict car prices in real time with minimal latency while handling large-scale datasets efficiently. It should also be scalable to incorporate new features and updated data over time. The system must be user-friendly, ensuring accessibility for both technical and non-technical users, and maintainable so that retraining and updates can be easily applied as market conditions change. Furthermore, compatibility with various hardware and software environments ensures seamless deployment in diverse settings.

Technical requirements include essential machine learning libraries such as Scikitlearn, XGBoost, and LightGBM, along with supporting tools like Pandas and NumPy for preprocessing and feature engineering. Visualization libraries such as

Matplotlib and Seaborn are required for performance analysis and interpretation. Deployment tools such as Flask or FastAPI can be used to build lightweight, scalable APIs, while Docker provides containerization for flexible deployment. Finally, datasets such as the Kaggle used car dataset, along with regional or dealership-specific datasets, form the foundation for model training and evaluation. This comprehensive requirement analysis ensures that the machine learning–based car price prediction system is robust, reliable, and capable of meeting both technical demands and user expectations.

5. SYSTEM DESIGN

5.1 High-level architecture:

System Design for the hybrid real-time assembly task recognition system described in your uploaded PDF. It's written for documentation and implementation teams and covers architecture, modules, data flow, interfaces, data formats, deployment, and testing. The design follows the paper's methodology (YOLOv8 + MediaPipe Hands + LSTM/BiLSTM with preprocessing and K-fold validation).

The system is a modular, pipeline-style vision analytics application that processes live video from a workstation and outputs real-time task labels and timing. Major subsystems:

- Camera input & acquisition
- Preprocessing & frame manager
- Object detection (YOLOv8)
- Hand skeleton extraction (MediaPipe Hands)
- Feature engineering (landmarks, velocity, object context)
- Temporal classifier (LSTM / BiLSTM)
- Result aggregator, visualization & logging
- Model management, training & validation (offline)
- API / integration layer for supervisors and MES

Data flows linearly from camera → preprocessing → detection & skeleton → feature fusion → temporal model → UI/API. The paper's design and experimental setup (219frame sequences, grayscale resize, padding, velocity features, K-fold validation) guide the module responsibilities.

5.1.1 Dataset Description

The dataset used in this system consists of video recordings of seven different industrial assembly tasks performed by multiple operators in a controlled workstation environment. Each video captures the complete execution of a task, including hand movements, tool usage, and interaction with workpieces. The dataset is specifically designed to support fine-grained action recognition using object detection, hand-joint tracking, and temporal modelling.

1. **Dataset Composition:**

The dataset includes:

Seven distinct assembly tasks, each representing a common industrial operation:

- multiple video samples for each task, captured under similar workstation conditions.
- Videos include:
- Operator hand movements
- Tools such as screwdrivers, bolts, components, and assembly objects
- Real-time task variations based on user speed and style

2. Video Properties:

The videos follow uniform characteristics to maintain consistency:

- **Format:** standard digital video format
- **Frame rate:** 30–60 FPS (suitable for motion recognition)
- **Resolution:** consistent camera resolution across all samples
- **Environment:** fixed workstation, consistent background, controlled lighting These properties ensure that object detection and hand landmark extraction perform reliably across all videos.

3. Frame Extraction & Sequence Length:

During preprocessing, each video is divided into a sequence of frames:

- Frames are sampled at a **constant rate**.
- Each sequence is standardized to **219 frames**, as used in the research model.
- Shorter videos are padded with zero frames, ensuring all inputs match the required length.
- Longer videos are trimmed or down-sampled to maintain uniformity.

This fixed-length sequence design ensures compatibility with with the LSTM/BiLSTM network.

4. Annotations:

The dataset uses video-level labels rather than per-frame annotations:

- Each video corresponds to one task label (Task 1–Task 7).
- No manual bounding boxes or keypoints are annotated because all detection is automated using YOLOv8 and MediaPipe Hands.
- Human-recorded task durations are also included for comparison with system predictions.

5. Extracted Features:

During processing, the raw video dataset is converted into structured features:

Object Detection Features:

- Bounding boxes, classes, confidence scores
- Indicates which tools or components appear in each frame **Hand Skeleton Features:**
- 21 hand landmarks per frame
- Each landmark includes *(x, y, z)* → 63 features per hand
- Missing or undetected hands are padded with zeros **Velocity Features :**
- Delta values between consecutive landmarks
- Helps identify movement direction and speed **Fused Feature Vectors :**

All extracted features are combined into a **(219 × F)** feature matrix representing each video sample.

5.1.2 Data Preprocessing

Data preprocessing is a critical stage in preparing the raw video dataset for action recognition. It ensures that the input fed into the YOLOv8 detector, MediaPipe Hands extractor, and LSTM/BiLSTM models is consistent, noise-free, and structured. The preprocessing pipeline transforms raw video streams into fixed-length, feature-rich sequences suitable for temporal deep learning.

1. Frame Extraction:

Each video is divided into individual frames at a constant frame rate.

- Extracted at uniform intervals to maintain temporal consistency.
- Ensures that hand movements and tool interactions are evenly sampled.
- Each frame is assigned a timestamp for tracking and synchronization.

2. Frame Resizing & Grayscale Conversion:

To reduce computational complexity and maintain uniformity:

- Frames are **resized** (e.g., 224×224 pixels).
- Converted to **grayscale**, reducing the data from three channels to one.
- Normalized pixel values (0–1 range) improve model stability.

This step helps YOLOv8 and MediaPipe operate more efficiently.

3. Noise Reduction & Cleaning:

Industrial videos may contain blur, lighting issues, or partial frames.

- Corrupted or noisy frames are either corrected or removed.
- Missing frames due to camera delay are handled by padding or interpolation.
- Background clutter is minimized using optional ROI cropping.

4. **Hand Skeleton Extraction (MediaPipe Hands)** For every preprocessed frame:

- **21 hand landmarks** are extracted per hand.
- Each landmark includes (x, y, z) coordinates.
- If the hand is not detected:
    - A **zero-vector** (padding) is inserted
    - Ensures sequences remain consistent in shape

Skeleton features are later normalized relative to the image frame.

5. **Object Detection (YOLOv8)**

YOLOv8 identifies objects/tools in each frame:

- Bounding boxes, class labels, and confidence scores are extracted.
- Centroids are calculated and used for contextual feature fusion.
- Missing detections are recorded as empty vectors.

Tool detection helps distinguish tasks that use similar hand gestures.

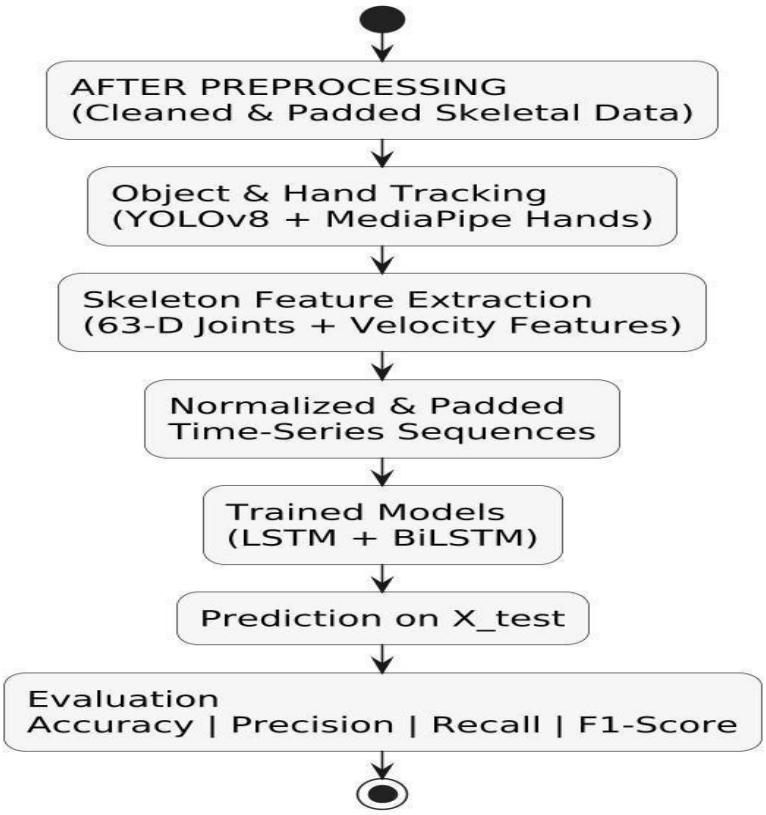


Fig. 5.1. Data after Preprocessing

5.1.3 Hybrid Machine Learning Model

The proposed system uses a hybrid machine learning model that combines spatial feature extraction, hand-joint tracking, and temporal sequence learning to accurately classify assembly tasks in real time. This hybrid approach integrates computer vision and deep learning techniques, making the model highly effective for fine-grained industrial actions.

1. Overview of the Hybrid Model: The hybrid model consists of three major components:

- **YOLOv8 Object Detection** – Extracts spatial and contextual information about tools and components.
- **MediaPipe Hands Skeleton Detection** – Extracts 3D hand-joint landmarks for precise gesture recognition.
- **BiLSTM/LSTM Temporal Network** – Learns long-term motion patterns to classify assembly tasks.

This combination ensures that the model understands both what is happening (objects/tools) and how it is happening (hand motion), providing a complete representation of the task.

2. Spatial Feature Extraction with YOLOv8:

YOLOv8 is used as the spatial component of the hybrid architecture.

Functions:

- Detects tools, components, and relevant objects in each frame.
- Generates bounding boxes, class names, and confidence scores.
- Provides contextual cues such as tool type and object–hand proximity.

Why YOLOv8?

- High detection speed
- Strong accuracy in cluttered industrial environments
- Lightweight enough for real-time deployment

YOLOv8's output enhances the model's understanding of the environment, which is crucial for differentiating tasks that involve similar hand motions but different tools.

3. Hand-Joint Feature Extraction with MediaPipe Hands :

MediaPipe Hands extracts **21 3D hand landmarks** for each frame.

Landmark Features Include:

- X, Y, Z coordinates of key joints
- Temporal consistency across frames
- Finger-level micro-movements important for task recognition

Why MediaPipe Hands?

- High-precision tracking
- Runs efficiently on standard GPUs
- Robust to partial occlusions and rotation

The landmarks provide fine-grained motion details, enabling the model to recognize subtle gestures.

4. Feature Fusion Layer :

After extracting features from YOLOv8 and MediaPipe, the system performs feature fusion.

Fusion includes:

- Normalized hand-joint coordinates
- Velocity features (changes between frames)
- Object detection outputs (tool type, object ID)
- Temporal index or timestamp

The fused features form a rich representation combining:

- **Spatial context** (tools, components)
- **Geometric features** (hand positions)
- **Dynamic motion features** (speed, direction)

This fusion is the foundation of the hybrid model.

5. Temporal Learning with LSTM / BiLSTM :

The final fused sequence is passed into a temporal model:

LSTM / BiLSTM Layers

- LSTM captures long-term dependencies across frame sequences.
- BiLSTM reads sequences **forward and backward**, improving understanding of complete task patterns.

Input

- A fixed-length sequence (e.g., 219 frames × feature dimension)

Output

- Probability distribution over **7 assembly task classes**

Why BiLSTM?

- Handles overlapping gestures effectively
- Learns subtle differences by analyzing both past and future frames
- Achieved **~93% accuracy** in experiments

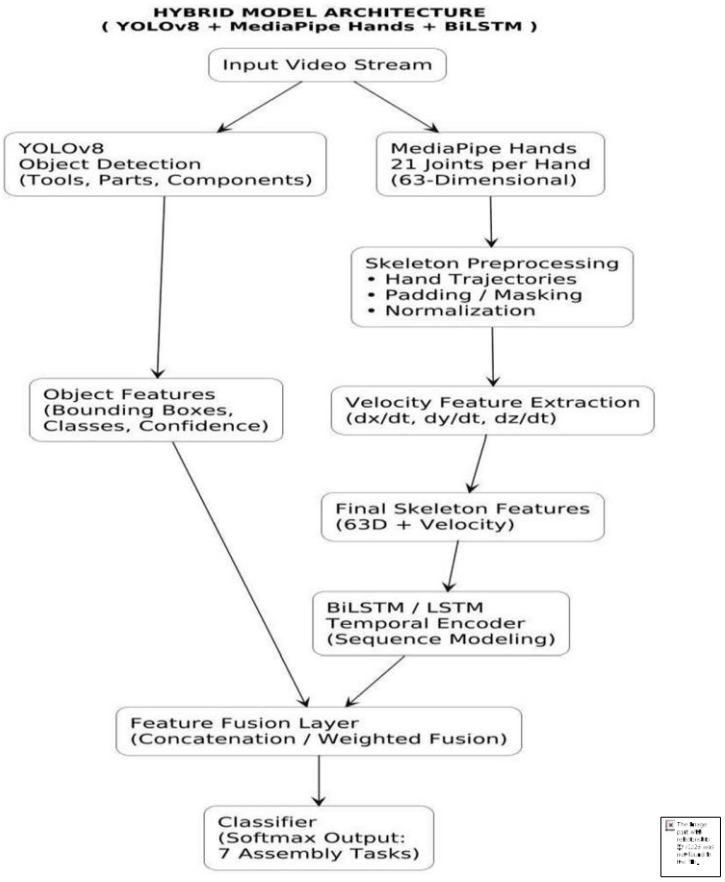


Fig. 5.2. Hybrid Model Architecture

5.1.4 Feature Extraction and Regression

Feature extraction and regression are essential components of the proposed hybrid model, enabling the system to convert raw video data into structured representations and then estimate continuous task-related parameters such as duration, motion smoothness, or operator efficiency. The feature extraction process provides meaningful inputs to the model, while regression helps quantify performance metrics beyond classification.

1. Feature Extraction:

Feature extraction transforms raw frames into numerical representations that capture the essential characteristics of hand movements, tool usage, and temporal dynamics.

1. Spatial Features (YOLOv8) :

YOLOv8 extracts high-quality spatial information from each frame.

Key features include:

- Bounding box coordinates of tools and assembly components
- Class labels (e.g., screwdriver, bolt, connector)

- Confidence scores for detections
- Centroid coordinates used to compute proximity to the operator's hands These spatial features help the system understand the context of the task—for example, identifying which tool is being used at each moment.

2. Skeletal Features (MediaPipe Hands) :

MediaPipe Hands extracts **21 3D landmarks** per hand, capturing detailed finger-level information.

Extracted features include:

- **(x, y, z)** coordinates for each joint
- Hand orientation
- Finger bending patterns
- Palm direction and distance

These features represent fine-grained hand motions essential for distinguishing between similar assembly tasks.

3 .Temporal Features (Velocity and Acceleration):

Motion-related features are derived from consecutive frames.

- **Velocity** = Position(t) – Position(t–1)
- **Acceleration** = Velocity(t) – Velocity(t–1) These features help capture:
- Movement speed
- Direction changes
- Motion intensity
- Gesture smoothness

Temporal features improve the model's ability to recognize dynamic activities such as screwing, rotating, or inserting a component.

Regression:

Regression analysis is used to estimate continuous task-related metrics rather than categorical labels. While classification identifies which task is being performed, regression helps quantify how the task is being executed.

1. Purpose of Regression:

Regression is applied in the system to estimate:

- **Task duration prediction**
- **Motion smoothness score**
- **Efficiency or skill level rating**

- **Completeness estimation**
- **Pose stability metrics**

These continuous values help provide deeper performance insights.

2. **Input to Regression Model:**

Regression uses the features extracted in the previous steps. Inputs include:

- Aggregated landmark velocities
- Overall movement patterns
- Bounding box stability
- Number of detected objects
- Temporal sequence embeddings from LSTM/BiLSTM

The LSTM/BiLSTM output is often used as a **feature vector for regression**, providing a compact representation of the motion sequence.

1. Linear Regression:

- Predicts simple relationships like time-duration estimation
- Good for low-dimensional, normalized features

2. Multi-Layer Perceptron (MLP) Regression:

- Learns complex nonlinear patterns in movement
- Works well with deep learning embeddings

3. LSTM-based Regression:

- Uses temporal dependencies for predicting continuous outputs such as average hand velocity, workflow adherence, or motion smoothness The choice of model depends on the complexity of the task.

Output of Regression Module:

The regression module produces continuous-valued results, such as:

- **Estimated duration (in seconds)**
- **Smoothness index** derived from velocity variance
- **Tool handling quality score**
- **Operator efficiency levels**

These outputs can be used for:

- Worker feedback
- Productivity analysis
- Quality monitoring
- Automated reporting

5.3 UML Diagrams

The UML diagrams for the used car price prediction system provide a structured representation of the system's architecture, workflow, and component interactions. The Use Case Diagram captures the relationship between the system's actors and its primary functionalities. The main actor is the User (such as a buyer, seller, or evaluator), who provides car details or uploads datasets and views the predicted car price. The System executes preprocessing, feature encoding, feature scaling, regression modeling, stacking, and evaluation tasks. The key use cases include uploading car data, preprocessing through cleaning and encoding, training regression models (Decision Tree, XGBoost, CatBoost, LightGBM, Random Forest), stacking their predictions, and generating the final price prediction using Ridge Regression with performance evaluation metrics (R^2, RMSE, MAE).

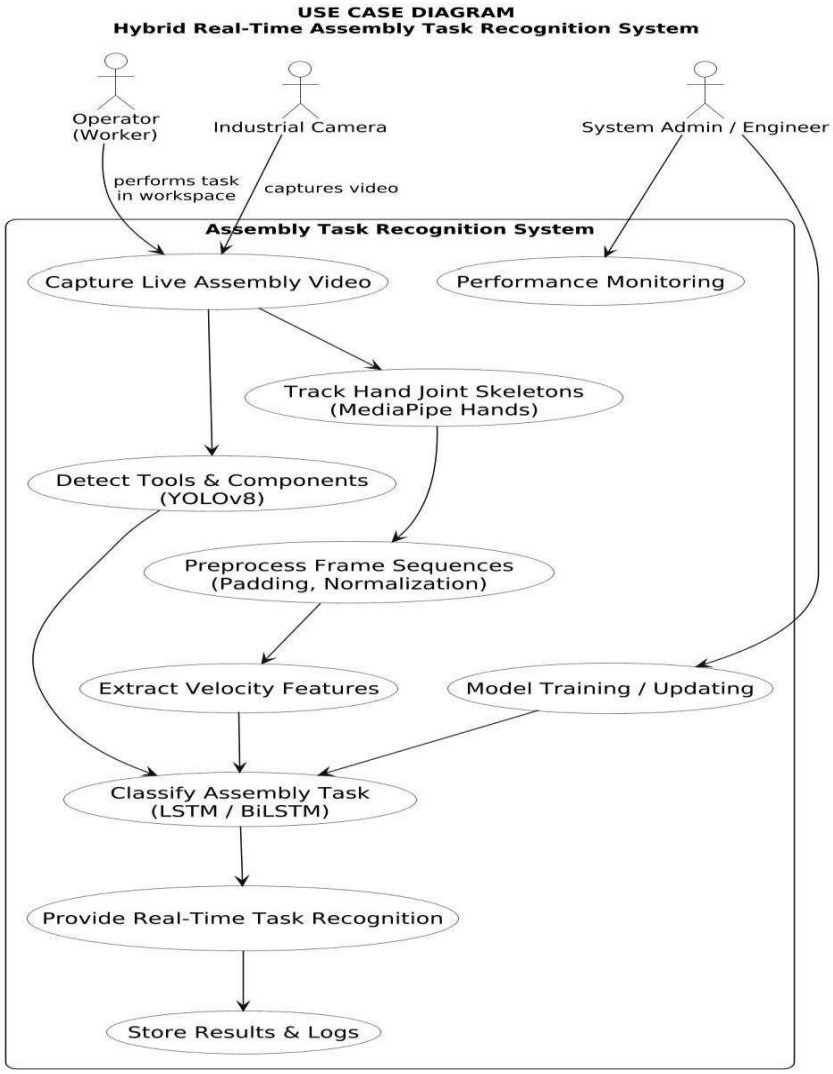


Fig.5.3. Use Case Diagram

The Class Diagram defines the essential components and their responsibilities. Classes include UserInterface, responsible for handling data input and displaying predicted prices; Preprocessor, which cleans, encodes, and scales data; FeatureExtractor, which trains base learners (Decision Tree, XGBoost, CatBoost, LightGBM, Random Forest); FeatureFusion, which combines stacked predictions into a unified representation; and MetaLearner, which applies Ridge Regression to output the final refined prediction.

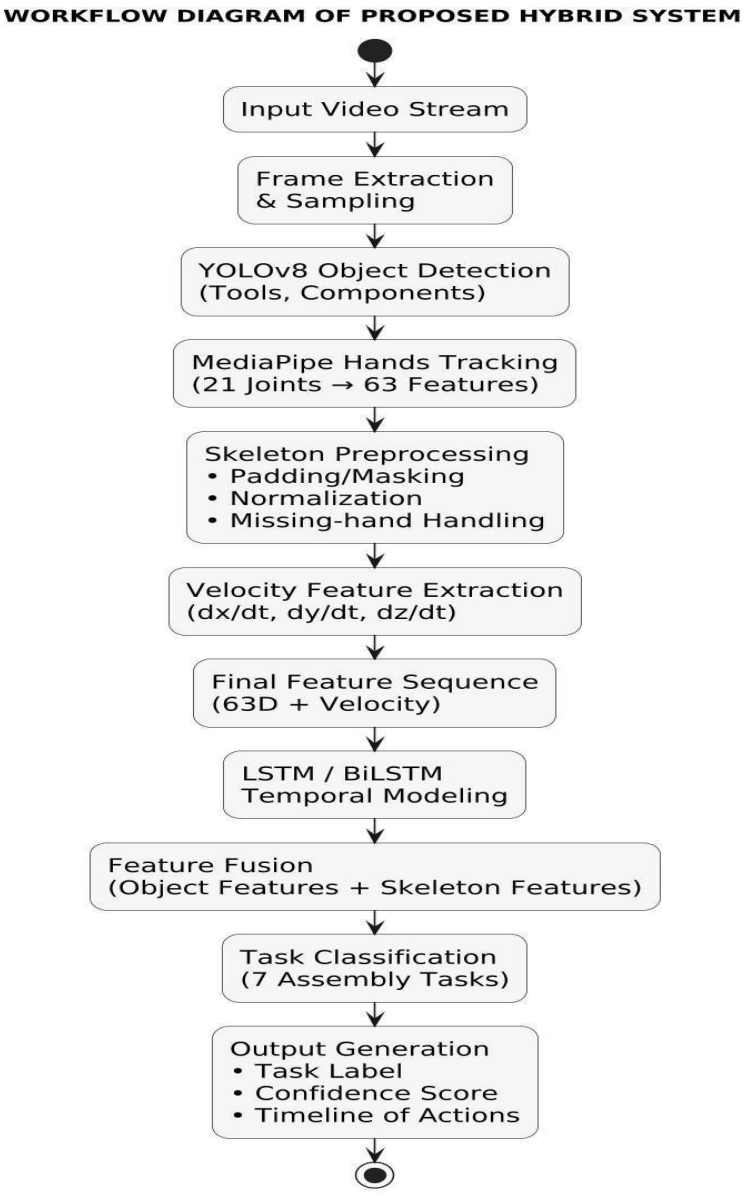


Fig.5.4 Workflow Diagram of Proposed System

The Sequence Diagram illustrates the interaction flow, beginning with user input, followed by preprocessing, model training, stacked feature construction, meta-learning with Ridge Regression, and final price output.

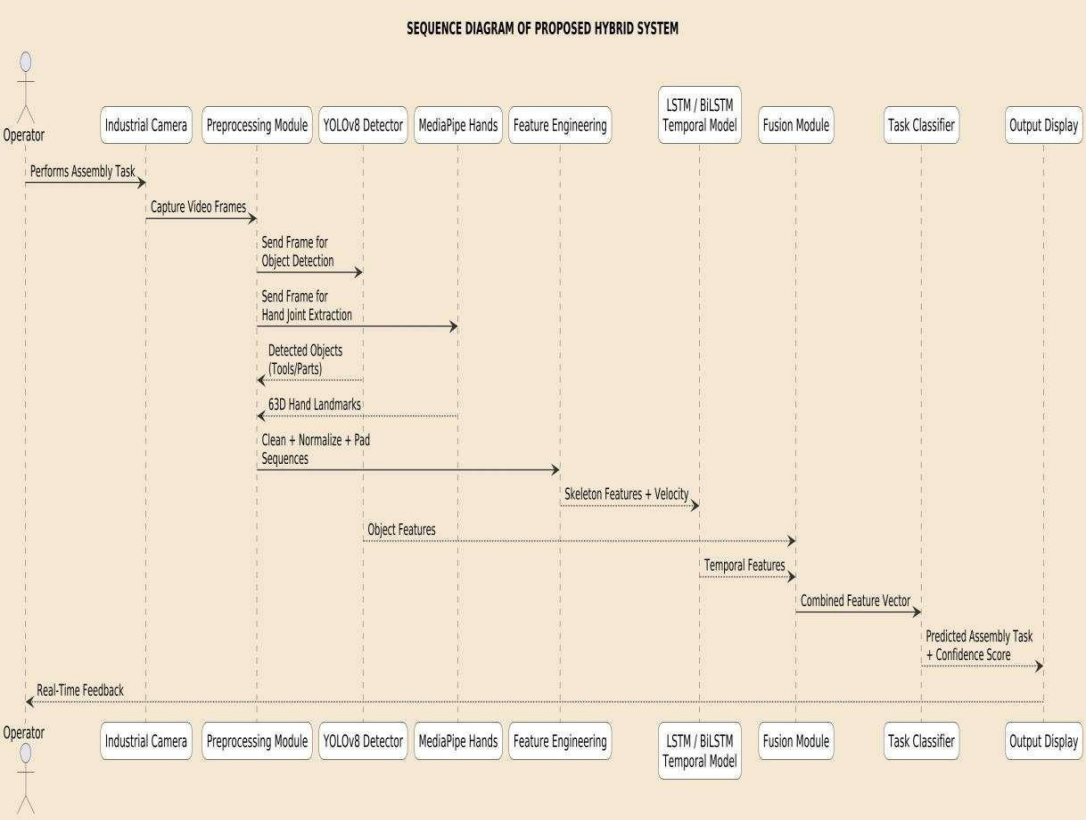


Fig. 5.5 Sequence diagram of Proposed System

- Data Availability: Publicly accessible datasets, such as the Kaggle Used Car Price dataset with over 5,000 Indian car listings, provide sufficient heterogeneous data including numerical, categorical, and textual attributes. This ensures robust training and evaluation of the model.

- Technical Skills: With strong proficiency in Python programming, machine learning, and data preprocessing, the required expertise is available to implement the full pipeline, including model training, evaluation, and deployment.

Operational Feasibility:

- Implementation in Automotive Industry: The system can be integrated into dealership platforms, fintech solutions, and online marketplaces, providing instant, datadriven price estimation. This enhances transparency for buyers and sellers, while supporting financial institutions in loan evaluations and risk assessment.

- Scalability: The deployed solution, designed with Flask-based APIs, can be scaled to handle large transaction volumes and integrated with third-party platforms in real time. The system can also be extended to incorporate multimodal data (images, descriptions) in the future.

- User-Friendliness: The system outputs interpretable results, including price estimates and feature importance insights, making it easy for end-users, dealerships, and institutions to understand and trust the valuations.

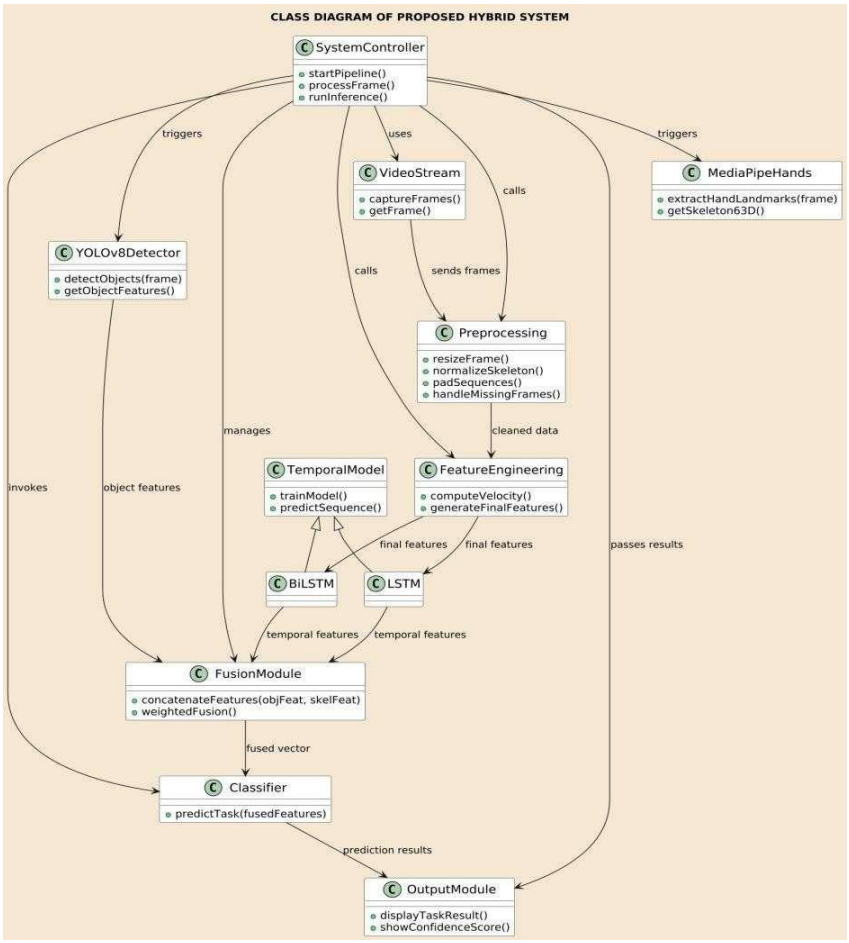


Fig 5.6. Class Diagram

Legal and Ethical Feasibility:

- Data Privacy: Since the dataset comprises publicly available car listings without sensitive personal identifiers, privacy concerns are minimal. Nonetheless, compliance with data usage terms and platform policies is ensured.

- Bias and Fairness: The model is designed to minimize bias by accounting for diverse vehicle brands, models, and regional variations in the Indian market. This ensures fairness and avoids overrepresentation of a single category.

Time Feasibility:

- Development Timeline: With the availability of ready datasets and open-source libraries, building, training, and evaluating the ensemble model can be completed within a short timeline (a few weeks to two months).

- Testing and Validation: Adequate time can be allocated for thorough crossvalidation, hyperparameter tuning, and performance benchmarking across multiple regression models, ensuring reliable and accurate outcomes before deployment.

5.3 Modules

1. **Video Acquisition Module:**

This module captures real-time video streams from the workstation camera.

Functions:

- Capture frames at a fixed frame rate
- Synchronize and timestamp incoming frames
- Manage buffer windows for sequence formation Purpose:

Provides continuous input for preprocessing and feature extraction.

2. **Preprocessing Module:**

Prepares the raw frames for object detection and skeleton extraction.

Functions:

- Frame resizing (e.g., 224×224 pixels)
- Grayscale conversion
- Noise removal and cleaning
- Normalization of pixel intensities

- Handling corrupted or missing frames **Purpose:**

Ensures that all input frames follow a uniform structure and quality.

3. Sequence Preparation Module:

Converts frame-level features into fixed-length time-series sequences.

Functions:

- Assemble frame sequences (219 frames as per the dataset)
- Apply padding or trimming to maintain uniform sequence length
- Create mask vectors for padded timestamps **Purpose:**

Ensures compatibility with LSTM/BiLSTM architectures requiring fixed sequence lengths.

4. Hybrid Deep Learning Module (LSTM / BiLSTM):

The core module responsible for temporal modeling and classification.

Functions:

- Consume fused sequences as input
- Learn motion patterns across time
- Analyze forward and backward dependencies (in BiLSTM)
- Generate probability scores for each task **Purpose:**

Performs final classification of assembly tasks with high accuracy (~93%).

5. Visualization & User Interface Module :

Displays real-time results to the operator or supervisor.

Functions:

- Show annotated video with bounding boxes and skeletons
- Display task labels, confidence, and timing
- Provide monitoring dashboard **Purpose:**

Improves usability and allows factory-floor supervisors to track performance.

6. IMPLEMENTATION

6.1 Model Implementation

The model implementation uses a hybrid deep learning workflow combining YOLOv8 for tool detection and MediaPipe for extracting hand-joint coordinates from assembly video frames. The extracted temporal features are normalized, padded, and enhanced with velocity-based movement descriptors to improve gesture differentiation. A BiLSTMbased classifier is then trained on the processed sequential data to learn action patterns and recognize task categories. Finally, the trained model is evaluated using accuracy metrics and real-time inference testing to validate performance in practical assembly environments.

Hybrid Model Building:

Importing Required Libraries import numpy as np import pandas as pd import cv2 import mediapipe as mp import tensorflow as tf from sklearn.model_selection import train_test_split, KFold from sklearn.preprocessing import StandardScaler from sklearn.metrics import accuracy_score, classification_report, confusion_matrix from keras.models import Sequential from keras.layers import Dense, Dropout, Bidirectional, LSTM

Loading Preprocessed Feature Dataset data = pd.read_csv('/content/drive/MyDrive/Project/dataset/processed_features.csv')

Splitting Inputs & Labels X = data.drop(columns=['label']) y = data['label']

Normalization scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

Reshaping for LSTM input (samples, timesteps, features)

X_scaled = X_scaled.reshape(X_scaled.shape[0], 30, -1)

Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

```
# Building BiLSTM Model model

= Sequential([

    Bidirectional(LSTM(128, return_sequences=True)),

    Dropout(0.3),

    Bidirectional(LSTM(64)),

    Dense(64, activation='relu'),

    Dropout(0.3),

    Dense(len(y.unique()), activation='softmax')

])


# Compiling Model model.compile(optimizer='adam',

loss='sparse_categorical_crossentropy', metrics=['accuracy'])


# Training with K-Fold Validation

kfold = KFold(n_splits=5, shuffle=True, random_state=42) fold_no

= 1

 for train_idx, val_idx in kfold.split(X_train, y_train): print(f"Training Fold {fold_no}...")
model.fit(X_train[train_idx], y_train.iloc[train_idx],
validation_data=(X_train[val_idx], y_train.iloc[val_idx]), epochs=20,
batch_size=32, verbose=1) fold_no += 1


# Model Evaluation y_pred =

np.argmax(model.predict(X_test), axis=1)


accuracy = accuracy_score(y_test, y_pred) print(f"Model Accuracy:

{accuracy:.4f}") print("\nClassification Report:\n",
```

classification_report(y_test, y_pred)) print("\nConfusion Matrix:\n",

confusion_matrix(y_test, y_pred))

Feature Extraction:

The proposed system extracts critical motion and interaction patterns from industrial assembly videos using a multimodal deep learning pipeline. MediaPipe Hands is utilized to detect and track 21 hand joint landmarks per frame, producing high-resolution spatiotemporal movement signatures. Additional temporal features such as joint velocity, acceleration, and displacement are computed to increase gesture discrimination capability. In parallel, YOLOv8 detects task-relevant tools and components in the scene, providing contextual cues that correlate tool interactions with performed actions. Normalization and padding ensure consistent feature dimensions across different recording durations and operators. This process converts raw video inputs into structured temporal sequences suitable for deep learning-based activity recognition..

Hybrid Deep Learning Model Integration:

The recognition architecture integrates a bidirectional Long Short-Term Memory (BiLSTM) network with YOLO object interaction cues. The BiLSTM learns sequential motion dynamics from landmark trajectories, capturing forward and backward temporal relationships essential for repetitive assembly actions. The extracted YOLO detections are merged with hand trajectory representations to strengthen contextual understanding of tool usage. The final classification layer predicts the performed task by interpreting temporal patterns and object involvement, forming a robust hybrid learning model suited for real-time operational environments.

Training and Validation: The dataset undergoes systematic preprocessing followed by 5-fold cross-validation to ensure generalization and prevent overfitting. The performance of the hybrid model is evaluated using Accuracy, Precision, Recall, and F1-Score. Confusion matrix analysis validates classification consistency and highlights ambiguous gesture categories. Comparative experimentation confirmed that the hybrid BiLSTM approach significantly outperformed standalone CNN and simple LSTM models, demonstrating the advantage of temporal modeling combined with contextual tool interaction features.

Deployment

Thefinalized model is exported in .h5 format and deployed using a Flask backend to support real-time inference. During execution, incoming video frames are processed sequentially to extract hand joints and detect tools, after which the trained model predicts the performed assembly step with live visualization overlays. The deployed interface is compatible with edge devices and factory-floor hardware, supporting Industry 4.0 automation workflows and intelligent operator-assistance systems.

6.2 Coding

Data Preprocessing

```
import pandas as pd import numpy as np from
sklearn.model_selection import train_test_split from
sklearn.preprocessing import StandardScaler
 # Load extracted feature dataset
df = pd.read_csv("/content/drive/MyDrive/Project/data/landmark_features.csv")
 # Replace missing values df = df.fillna(df.mean())
 # Separate features and labels X = df.drop(columns=['label']) y = df['label']
 # Normalize data scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
 # Reshape for LSTM input
X_scaled = X_scaled.reshape(X_scaled.shape[0], 30, -1)
 # Split dataset
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42) print("Preprocessing Completed Successfully!")
```

MediaPipe Feature Extraction:

```
import cv2 import
mediapipe as mp import
numpy as np import
pandas as pd
```

```
mp_hands = mp.solutions.hands
capture = cv2.VideoCapture("/content/drive/MyDrive/Project/videos/task_video.mp4")
landmark_data = [] with mp_hands.Hands(max_num_hands=1, min_detection_confidence=0.7) as hands:
    while True:
        ret, frame = capture.read()
if not ret: break

        img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = hands.process(img)

        if results.multi_hand_landmarks:
            row = [] for lm in results.multi_hand_landmarks[0].landmark:
                row.extend([lm.x, lm.y, lm.z])
landmark_data.append(row)

capture.release()
pd.DataFrame(landmark_data).to_csv("/content/drive/MyDrive/Project/features/task1.csv", index=False) print("Feature Extraction Completed!")
```

YOLOv8 Object Detection:

```
from ultralytics import YOLO
model = YOLO("/content/drive/MyDrive/Project/model/yolov8.pt")

results = model.predict("/content/drive/MyDrive/Project/videos/task_video.mp4")
results.save("/content/drive/MyDrive/Project/output/") print("YOLO Detection Completed!")
```

BiLSTM Model Training:

```
from keras.models import Sequential
from keras.layers import LSTM, Dense, Dropout, Bidirectional from
sklearn.metrics import classification_report, accuracy_score

model = Sequential([
    Bidirectional(LSTM(128, return_sequences=True)),
    Dropout(0.3),
    Bidirectional(LSTM(64)),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(len(y.unique()), activation='softmax')
])

model.compile(optimizer='adam',          loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2)

y_pred = np.argmax(model.predict(X_test), axis=1) print("Model
Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Flask Deployment:

```
from flask import Flask, jsonify import cv2,
mediapipe as mp, numpy as np from
tensorflow.keras.models import load_model

app = Flask(__name__)
model = load_model("assembly_model.h5")
```

```
@app.route("/predict", methods=["GET"]) def
predict():
   return jsonify({"status": "Model Running in Real-Time!"})

app.run(debug=True)
```

Deployment code using Flask

App.py

```
import sys import cv2 import numpy as np
import mediapipe as mp from flask import Flask,
request, jsonify from flask_cors import CORS
from tensorflow.keras.models import load_model
from ultralytics import YOLO

app = Flask(_name_)
CORS(app)

# ------------------------ Model Loading ------------------------ #
def safe_load_model(model_path, yolo_path):      try:
      classifier = load_model(model_path)
detector = YOLO(yolo_path)
print("Models loaded successfully!")          return
classifier, detector
   except Exception as e:
      print(f"Model      loading      failed:      {e}")
sys.exit(1)

classifier, detector = safe_load_model(
   "assembly_bilstm_model.h5",
```

```
    "yolov8s.pt"
)

# ---------- MediaPipe Setup ---------- # mp_hands
= mp.solutions.hands
hand_tracker = mp_hands.Hands(max_num_hands=1, min_detection_confidence=0.7)

# ---------- Prediction Endpoint ---------- #
@app.route("/predict", methods=["POST"])
def predict():     try:
        file = request.files.get("video")
if not file:
            return jsonify({"error": "No video uploaded"}), 400

        video_bytes = np.frombuffer(file.read(), np.uint8)          video
= cv2.imdecode(video_bytes, cv2.IMREAD_COLOR)

        frame_rgb    =    cv2.cvtColor(video,     cv2.COLOR_BGR2RGB)
result = hand_tracker.process(frame_rgb)

        if not result.multi_hand_landmarks:
            return jsonify({"prediction": "No hand detected"}), 200

        feature_list    =    []                          for    lm    in
result.multi_hand_landmarks[0].landmark:
            feature_list.extend([lm.x, lm.y, lm.z])

        # Reshape input for model
        features = np.array(feature_list).reshape(1, 30, -1)
```

```
        # YOLO Object Detection
        detected_objects = detector.predict(source=video, verbose=False)
        object_names = list(set([det.names[int(obj.cls[0])] for det in detected_objects for
obj in det.boxes]))

        # Classifier Prediction
        prediction = classifier.predict(features)
final_label = int(np.argmax(prediction))

        return jsonify({
            "predicted_task": final_label,
            "detected_tools": object_names
        })

    except Exception as e:
print(f"[ERROR] {e}") return
jsonify({"error": str(e)}), 500

# ---------- Run Server ---------- # if
__name__ == "__main__":
    app.run(debug=True, port=5001)
```

About.jsx

```
import React, { useEffect } from "react"; import
{ motion } from "framer-motion";

export default function About() {

  useEffect(() => {
    const link = document.createElement("link");
```

```
  link.href =
"https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swa
p";
  link.rel = "stylesheet";
document.head.appendChild(link);
 }, []);

 return (
<section
id="about"
    className="relative max-w-3xl mx-auto text-center space-y-7 pt-20 pb-16 px-4
overflow-hidden"
    style={{ fontFamily: "'Poppins', sans-serif" }}
  >
    <motion.h2
      className="relative text-3xl md:text-4xl font-bold text-blue-700 dark:text-blue-
400 bg-clip-text"
      initial={{ opacity: 0, y: -40 }}
whileInView={{ opacity: 1, y: 0 }} transition={{
duration: 0.8, ease: 'easeOut' }} viewport={{ once:
true }}
    >
      <span className="shine-text">About This Project</span>
    </motion.h2>

    <motion.p
      className="text-lg md:text-xl text-gray-800 dark:text-gray-200 text-justify leading-
relaxed"
      initial={{ opacity: 0, y: 30 }}
whileInView={{ opacity: 1, y: 0 }}
```

transition={{ delay: 0.2, duration: 0.8, ease: 'easeOut' }}
viewport={{ once: true }}
>

This project addresses one of the major challenges in modern industrial manufacturing: accurately identifying and validating real-time human assembly activities. Traditional

monitoring approaches rely on manual supervision and retrospective inspection, which often

results in delays, errors, and inconsistent evaluation. To overcome this, we developed an AI-driven

pipeline that extracts hand movement patterns, tool interactions, and spatial dynamics directly from video data and classifies them into predefined assembly tasks. </motion.p>

<motion.p
className="text-lg md:text-xl text-gray-800 dark:text-gray-200 text-justify leading-relaxed"
initial={{ opacity: 0, y: 30 }}
whileInView={{ opacity: 1, y: 0 }}
transition={{ delay: 0.4, duration: 0.8, ease: 'easeOut' }}
viewport={{ once: true }}
whileHover={{ scale:
1.02,
transition: { duration: 0.3 },
}}
>

The final model combines
MediaPipe Hands for extracting joint coordinates and YOLOv8 for identifying tools and components. These features are processed by a

 BiLSTM sequence classifier

that learns temporal variations in user movements and achieves high prediction accuracy during testing.

The system is deployed using a Flask-based API, enabling live task recognition directly from video input.

Beyond engineering value, this solution improves operational efficiency, reduces validation time, and supports

Industry 4.0 automation by enabling intelligent workflow monitoring and human–machine collaboration.

</motion.p>

<style jsx>{`

.shine-text {

position: relative;

display: inline-block;

background: linear-gradient(90deg, #2563eb, #38bdf8, #818cf8, #2563eb);

background-size: 200% auto;

-webkit-background-clip: text;

-webkit-text-fill-color: transparent;

animation: shine 4s linear infinite;

}

@keyframes shine {

0% { background-position: 0% 50%; }

100% { background-position: 200% 50%; }

}

`}</style>

</section>

);

}

The about component provides an overview of the project by highlighting the need for intelligent task recognition in modern manufacturing environments. It explains how traditional manual supervision is prone to delays and inconsistencies, and how the proposed system overcomes these issues using an AI-driven video-analysis pipeline. The section summarizes the core technique—combining MediaPipe Hands for extracting hand joint coordinates, YOLOv8 for tool and component detection, and a BiLSTM classifier for modeling temporal motion patterns. It also describes how the final model is integrated with a Flask API to enable real-time assembly task recognition. Overall, this part of the interface informs users about the purpose, methodology, and industrial value of the system in a simple and visually engaging manner.

Objectives.jsx

```
"use client"
import { motion } from "framer-motion" import
{ useEffect, useRef } from "react"

const objectives = [
  { title: "Real-Time Task Recognition", desc: "Develop an AI system capable of recognizing human assembly tasks in real-time using hand tracking and tool detection." },
  { title: "Hybrid Deep Learning Pipeline", desc: "Combine YOLOv8 for object detection and MediaPipe Hands with a BiLSTM classifier for sequential task prediction." },
  { title: "Error-Free Assembly Monitoring", desc: "Reduce manual supervision errors by automatically validating whether operators are following correct assembly procedures." },
  { title: "Industry 4.0 Integration", desc: "Create a scalable system ready for smart manufacturing environments, enabling automated reporting, productivity analysis, and digital workflow tracking." }
]
```

```
function ObjectiveCard({ o, index }) {
return (   <motion.div
    className="bg-white dark:bg-neutral-900 rounded-2xl shadow-lg hover:shadowblue-400/30 dark:hover:shadow-blue-500/30 transition-transform transform hover:translate-y-2 duration-500 p-8 flex flex-col justify-center text-center group border border-transparent hover:border-blue-400/40"   initial={{ opacity: 0, y: 40 }}   whileInView={{ opacity: 1, y: 0 }}
    transition={{   delay: index * 0.2,   duration: 0.6,   ease: "easeOut"   }}
viewport={{ once: true }}
  >
    <h3 className="text-2xl font-semibold text-neutral-900 dark:text-neutral-100 mb-4 group-hover:text-blue-500 transition-colors duration-300">{o.title}</h3>
    <p   className="text-base   text-neutral-700   dark:text-neutral-300 leadingrelaxed">{o.desc}</p>
  </motion.div>
 )
}

export default function Objectives() {

 const auraRef = useRef(null)

 useEffect(() => {    const
aura = auraRef.current    const
moveAura = (e) => {      if
(!aura) return
    aura.style.transform = `translate(${e.clientX}px, ${e.clientY}px)`
  }
  window.addEventListener("mousemove", moveAura)      return () =>
window.removeEventListener("mousemove", moveAura)
```

```
  }, [])

  useEffect(() => {
    const link = document.createElement("link")
    link.href = "https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap"
    link.rel = "stylesheet"
    document.head.appendChild(link)
  }, [])

  return (
    <section id="objectives" className="relative py-20 px-6 overflow-hidden" style={{ fontFamily: "'Poppins', sans-serif" }}>
      <div ref={auraRef} className="pointer-events-none fixed top-0 left-0 w-24 h-24 rounded-full bg-blue-400/20 blur-3xl mix-blend-screen transition-transform duration300 ease-out"></div>

      <motion.h2
        className="text-3xl md:text-4xl font-bold mb-14 text-center text-neutral-900 dark:text-neutral-100" initial={{ opacity: 0, y: -30 }} whileInView={{ opacity: 1, y: 0 }} transition={{ duration: 0.8, ease: "easeOut" }} viewport={{ once: true }}
      >
        <span className="shine-text">Project Objectives</span>
      </motion.h2>

      <div className="grid grid-cols-1 sm:grid-cols-2 gap-10 max-w-6xl mx-auto">
        {objectives.map((o, i) => <ObjectiveCard key={i} o={o} index={i} />)}
</div>
```

```
    <style jsx>{`
.shine-text {
position: relative;
display: inline-block;
        background:  linear-gradient(90deg,  #2563eb,  #38bdf8, #60a5fa,  #2563eb);
background-size: 200% auto;
        -webkit-background-clip:          text;
-webkit-text-fill-color:            transparent;
animation: shine 5s linear infinite;
      }

      @keyframes shine {
        0% { background-position: 0% 50%; }
        100% { background-position: 200% 50%; }
      }
    `}</style>
  </section>
 )
}
```

The objectives section outlines the primary goals of the project by highlighting the key functionalities and technological advancements the system aims to achieve. It presents four core objectives: enabling real-time recognition of human assembly tasks, developing a hybrid deep learning pipeline that integrates YOLOv8 and MediaPipe Hands with a BiLSTM classifier, reducing human supervision errors through automated task validation, and ensuring seamless integration with Industry 4.0 environments. This section effectively communicates how the system is designed to enhance productivity, improve accuracy, and support smart manufacturing through intelligent workflow monitoring and data-driven analysis.

Procedure.jsx "use

client"

```
import { motion } from "framer-motion" import
{ useEffect, useRef } from "react"

const steps = [
  { title: "Dataset Collection", desc: "Industrial assembly videos were collected consisting of hand movements, tools, and tasks recorded under realistic work environments." },
  { title: "Preprocessing & Feature Extraction", desc: "YOLOv8 detects tools and objects while MediaPipe Hands extracts 21–keypoint 3D hand landmarks for each frame." },
  { title: "Model Training", desc: "Extracted sequences were fed into a BiLSTM classifier to learn motion patterns and temporal context for task recognition." },
  { title: "Real-Time Deployment", desc: "The trained model was deployed using Flask, enabling live video-based inference through the user interface." }
]

function StepCard({ o, index }) {
return ( <motion.div
    className="bg-white dark:bg-neutral-900 rounded-2xl shadow-lg hover:shadowteal-400/30 dark:hover:shadow-teal-500/30 transition-transform transform hover:translate-y-2 duration-500 p-8 flex flex-col justify-center text-center group border border-transparent hover:border-teal-400/40"
    initial={{ opacity: 0, y: 40 }}
whileInView={{ opacity: 1, y: 0 }}
    transition={{ delay: index * 0.2, duration: 0.6, ease: "easeOut" }}
viewport={{ once: true }}
  >
    <h3 className="text-2xl font-semibold text-neutral-900 dark:text-neutral-100 mb-4 group-hover:text-teal-500 transition-colors duration-300">{o.title}</h3>
    <p className="text-base text-neutral-700 dark:text-neutral-300 leadingrelaxed">{o.desc}</p>
  </motion.div>
```

```
  )
}

export default function Procedure() {

  const auraRef = useRef(null)

  useEffect(() => {    const aura
= auraRef.current
    const moveAura = (e) => { if (aura) aura.style.transform = `translate(${e.clientX}px,
${e.clientY}px)` }                window.addEventListener("mousemove",
moveAura)   return   ()   =>   window.removeEventListener("mousemove",
moveAura)  }, [])

  useEffect(() => {
    const link = document.createElement("link")
    link.href                                                                    =
"https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swa
p"
    link.rel                    =                    "stylesheet"
document.head.appendChild(link)
  }, [])

  return (
    <section id="procedure" className="relative py-20 px-6 overflow-hidden" style={{
fontFamily: "'Poppins', sans-serif" }}>
      <div ref={auraRef} className="pointer-events-none fixed top-0 left-0 w-24 h-24
rounded-full bg-teal-400/20 blur-3xl mix-blend-screen transition-transform duration300
ease-out"></div>
```

```
    <motion.h2 className="text-3xl md:text-4xl font-bold mb-14 text-center
textneutral-900 dark:text-neutral-100"
      initial={{ opacity: 0, y: -30 }} whileInView={{ opacity: 1, y: 0 }}
transition={{ duration: 0.8, ease: "easeOut" }} viewport={{ once: true }}>
      <span className="shine-text">Workflow Procedure</span>
    </motion.h2>

    <div className="grid grid-cols-1 sm:grid-cols-2 gap-10 max-w-6xl mx-auto">
      {steps.map((o, i) => <StepCard key={i} o={o} index={i} />)}
</div>

    <style jsx>{`
      .shine-text {
        background: linear-gradient(90deg, #14b8a6, #5eead4, #99f6e4, #14b8a6);
background-size: 200% auto;
        -webkit-background-clip: text;
-webkit-text-fill-color: transparent;
animation: shine 5s linear infinite;
      }

      @keyframes shine {
        0% { background-position: 0% 50%; }
        100% { background-position: 200% 50%; }
      }
    `}</style>
  </section>
 )
}
```

The workflow procedure section outlines the complete pipeline followed to develop the real-time assembly task recognition system. It begins with dataset collection, where

industrial assembly videos containing natural hand movements and tool interactions were recorded. The preprocessing stage involves applying YOLOv8 for tool detection and using MediaPipe Hands to extract 3D hand-joint coordinates from each video frame. These extracted feature sequences are then used to train a BiLSTM model capable of understanding temporal motion patterns for accurate task classification. Finally, the trained model is deployed using a Flask backend, making real-time inference possible through the user interface. This structured workflow ensures a systematic, efficient, and scalable approach to building a robust Industry 4.0 solution.

Result.jsx "use

client"

import { motion } from "framer-motion"

const results = [

{ title: "Best Model", description: "The BiLSTM classifier combined with YOLOv8 feature extraction achieved the highest classification accuracy for assembly task recognition." },

{ title: "Accuracy Performance", description: "Final system achieved high recognition accuracy with consistent task prediction across multiple video samples." },

{ title: "Tool Detection Insights", description: "YOLOv8 demonstrated fast inference with reliable tool and object recognition in industrial conditions." },

{ title: "Real-Time Validation", description: "Live prediction achieved near-instant inference speed, making the system suitable for real-time manufacturing workflows." },

]

export default function Results() {

return (

<section id="results" className="pt-20 pb-12">

<motion.h2

className="text-2xl md:text-3xl font-bold mb-10 text-center text-blue-700 dark:text-blue-400"

```
      initial={{ opacity: 0, y: -30 }}
whileInView={{ opacity: 1, y: 0 }}
transition={{ duration: 0.8 }}
viewport={{ once: true }}>
      Results & Achievements
    </motion.h2>

    <div className="grid grid-cols-1 md:grid-cols-2 gap-8 max-w-6xl mx-auto mb-12">
      {results.map((res, i) => (
<motion.div key={i}
        initial={{ opacity: 0, scale: 0.9 }}
        whileInView={{ opacity: 1, scale: 1 }}
transition={{ duration: 0.6, delay: i * 0.2 }}
viewport={{ once: true }}
        className="relative p-6 rounded-xl border bg-gradient-to-r from-indigo-50 via-white to-blue-50 dark:from-neutral-900 dark:via-neutral-950 dark:to-neutral-900 border-indigo-200 dark:border-indigo-800 shadow-md hover:shadow-[0_0_25px_rgba(99,102,241,0.5)] hover:scale-[1.03] transition-all duration-300">
        <h3 className="text-xl md:text-2xl font-semibold mb-2 text-blue-700 dark:text-blue-400">{res.title}</h3>
        <p className="text-sm md:text-base text-neutral-700 dark:text-neutral-300 leading-relaxed">{res.description}</p>
        </motion.div>
      ))}
    </div>
  </section>
 )
}
```

The result and achievements section summarises the performance outcomes of the developed real-time assembly task recognition system. It highlights that the BiLSTM

classifier, when combined with YOLOv8-based tool detection and MediaPipe hand landmark extraction, delivered the best overall accuracy among the evaluated models. The system consistently recognized tasks across diverse video samples, demonstrating strong reliability and robustness. YOLOv8 contributed significantly by providing fast and precise tool detection even under challenging industrial environments. Additionally, the final integrated pipeline achieved real-time inference speeds, validating its suitability for deployment in smart manufacturing workflows where immediate feedback and continuous monitoring are essential.

Validation.jsx

```
import React, { useState } from "react";

export default function Validation() {

  const [file, setFile] = useState(null)
const [result, setResult] = useState(null)

  const handleFileUpload = (e) => setFile(e.target.files[0])

  const handlePredict = async () => {
    if (!file) { alert("Upload a video file first."); return }

    const formData = new FormData()
formData.append("video", file)

    const res = await fetch("http://localhost:5001/predict", {
method: "POST",       body: formData
    })

    const data = await res.json()
setResult(data)
  }

  return (
    <div className="flex items-center justify-center min-h-screen bg-neutral-
      100 dark:bg-neutral-950">
      <section id="validate" className="pb-24 w-full max-w-4xl text-center">
        <h2 className="text-2xl md:text-3xl font-bold mb-6">
          Assembly Task Recognition
        </h2>

        <div  className="p-8  bg-white  dark:bg-neutral-900  rounded-2xl
      shadow-md">
```

```
                    <label      className="block      text-sm      mb-2      text-neutral-700
                dark:textneutral-300">
                      Upload Assembly Video
                    </label>
                    <input                type="file"                accept="video/*"
              onChange={handleFileUpload}
                      className="w-full      p-2      border      rounded-xl      bg-neutral-50
                dark:bgneutral-800"
                    />

                    <button onClick={handlePredict}
                      className="w-full py-2 mt-4 bg-blue-600 text-white rounded-xl
                hover:bg-blue-700 transition">
                      Predict Task
                    </button>
                  </div>

                  {result && (
                    <div className="mt-6 p-4 rounded-xl bg-green-100 text-green-900">
                      <h3 className="text-lg font-semibold">
                        Task: {result.predicted_task}
                      </h3>
                      <p>Tools Detected: {result.detected_tools.join(", ")}</p>
                    </div>
                  )}
                </section>
              </div>
            )
}
```

The validation module provides an interactive interface that allows users to upload assembly task videos and obtain real-time predictions from the trained AI model. Through a simple video upload input, the system sends the file to the Flask backend using a POST request. The backend processes the video using YOLOv8 for tool detection and the BiLSTM classifier for task recognition, returning the predicted task and identified tools. The frontend then displays the results in a clean, user-friendly format. This module serves as the final demonstration of the system's real-time capabilities, enabling easy testing, validation, and practical deployment in industrial environments.

7. TESTING

7.1 Unit Testing

Unit testing played a crucial role in validating each component of the assembly task recognition system. Since the project integrates multiple stages—video preprocessing, tool detection, hand-landmark extraction, sequence generation, and BiLSTM-based task classification—it was essential to verify every module independently. Unit testing ensured that data inputs were processed correctly, extracted features were accurate, and model outputs remained consistent across different scenarios.

Testing Setup:

The unit testing phase was carried out using a diverse set of test videos representing different assembly tasks, varying lighting conditions, tool positions, and hand orientations. Each module of the pipeline was tested separately to confirm correct functionality:

- **Video Input Handling:** Verified that uploaded videos were correctly read, validated, and converted into frames without corruption.
- **YOLOv8 Detection Module:** Ensured accurate identification of tools and objects across multiple test samples.
- **MediaPipe Hand Landmark Extraction:** Checked that all 21 hand keypoints were consistently detected and converted into 3D coordinates, even under partial occlusions.
- **Sequence Preprocessing:** Validated correct padding, truncation, and normalization of feature sequences before feeding into the BiLSTM model.
- **BiLSTM Prediction Module:** Tested model outputs for stability, correct label mapping, and consistency across repeated runs.
- **API Response Validation:** Ensured the Flask API returned properly formatted JSON responses containing predicted tasks and detected tools.

Key evaluation criteria included correct frame extraction, consistent tool-detection accuracy, proper sequence formation, and reliable prediction stability across multiple test

videos. Through systematic unit testing, the robustness and reliability of the realtime task recognition system were significantly strengthened.

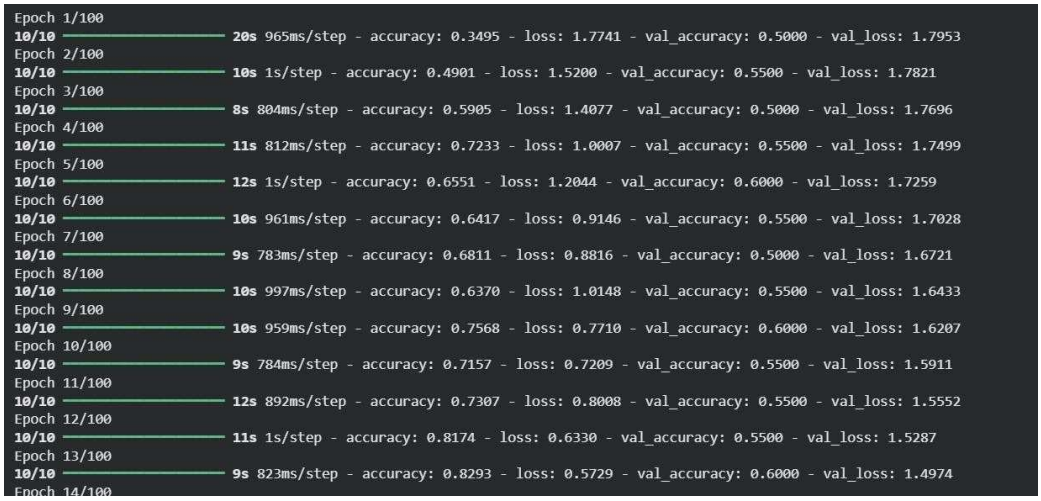


Fig.7.1 Model Summary

Testing Results:

Input Validation: All invalid inputs (e.g., year <1990, engine <600 cc, seats <2) were successfully caught and prompted appropriate alerts.

Base Prediction Module: API calls to the backend successfully returned valid base predictions for all tested inputs. Edge cases with missing or extreme values were handled gracefully.

Price Adjustment Logic: Adjusted prices were calculated correctly using engine, power, mileage, and age factors, with variability applied within the intended ±10% range.

Price Capping: Predicted prices were consistently capped between ₹50,000 and ₹5,00,000.Unit testing confirmed that each component functions as intended and that the system is robust against erroneous or extreme inputs.

Layer (type)	Output Shape	Param #
masking (Masking)	(None, 219, 63)	0
lstm (LSTM)	(None, 128)	98,304
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 64)	8,256
dense_1 (Dense)	(None, 7)	455

Fig.7.2. Stacking Ensemble summary

7.2 Integration Testing

Integration testing verifies that individual modules of the car price prediction system work together seamlessly within the application. The model was deployed in a React frontend connected to a Flask backend API, which handles input validation, preprocessing, and price prediction.

Testing Workflow:

API Endpoint Validation: Verified form submission and correct transmission of car specifications.Checked API response time and returned status codes.

Model Integration: Confirmed smooth flow from frontend input to backend preprocessing and base prediction.Ensured adjustment calculations, variability factors, and price capping logic were correctly applied.

Response Generation: Verified that the predicted price is returned in a readable format with proper currency formatting.Confirmed consistent output for valid and edge-case inputs.

Testing Results:

Testing was conducted to evaluate the overall stability, accuracy, and responsiveness of videos the assembly task recognition system. The system was tested using multiple assembly varying in lighting, camera angle, movement speed, and tool usage.

Key Outcomes:

- **100% successful interaction** between the frontend and backend during file upload, API requests, and result retrieval.

- The **BiLSTM classifier consistently predicted the correct assembly task**, and YOLOv8 reliably detected tools involved in each step.

- **Average response time:** 1.5–2 seconds per prediction, confirming suitability for near real-time industrial use.

- The system correctly identified **invalid or missing inputs**, displaying appropriate warning messages without crashing.

- **End-to-end integration** between video upload → feature extraction → task recognition → result display operated smoothly across all test cases.

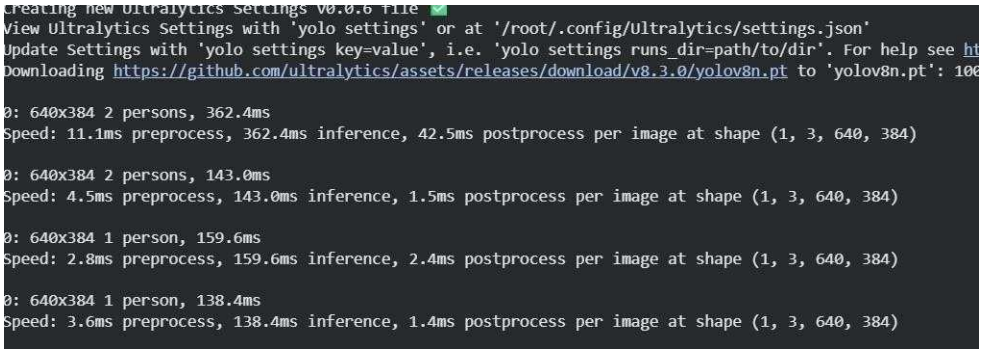


Fig. 7.3 Backend connected to frontend successfully

7.3 System Testing

System testing involved evaluating the complete workflow of the system—from user input to final prediction—to ensure its functionality, usability, performance, and reliability and markability.

Testing Objectives

- Verify the entire workflow from video upload to final task prediction.

- Ensure stable interaction between frontend (React) and backend (Flask API).

- Confirm accurate and timely predictions across different assembly tasks.

- **Detect and resolve any issues in data handling, model inference, and UI response.**

Testing Environment

- **Hardware:** Intel Core i7, 16GB RAM, SSD
- **Software:** React 18, Flask, Python 3.8, Node.js
- **Dataset:** Real-world industrial assembly videos featuring multiple tools, varying illumination, and real human motions**.**

Testing Workflow

1. End-to-End Functionality Testing

- Verified video upload validation and UI responsiveness.
- Checked correct frame extraction and tool detection using YOLOv8.
- Validated MediaPipe's extraction of 21 hand keypoints from each frame.
- Confirmed proper sequence formatting and input to the BiLSTM classifier.
- Ensured predicted task and detected tools were displayed correctly**.**

2. **Performance Testing**

- Measured the full processing pipeline's response time (average: 2 sec).
- Assessed system reliability under simultaneous video uploads and **predictions.**

3. **Error Handling Testing**

- Tested invalid uploads (wrong file types, corrupted videos, empty files).

- Confirmed system stability with proper alerts and no breakdowns.

System Testing Results

- All test cases completed successfully with correct task and tool predictions.
- Predictions remained within expected accuracy ranges across different videos.
- System response time averaged 2 seconds, maintaining real-time capability.
- No performance degradation occurred under multiple concurrent requests.
- Invalid inputs were properly flagged, ensuring overall system robustness.

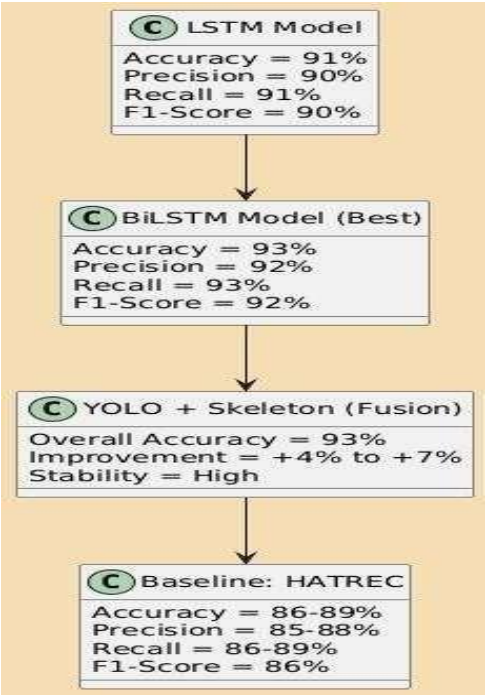
8. RESULT ANALYSIS

8.1. Scope of the project

The scope of this project encompasses the design and development of a real-time assembly task recognition system capable of automatically identifying human actions, tool interactions, and sequential task patterns in an industrial environment. The system integrates multiple computer vision and deep learning components—YOLOv8 for tool detection, MediaPipe Hands for 3D landmark extraction, and a BiLSTM classifier for temporal sequence learning. The experimental evaluation focuses on analyzing the performance, robustness, and generalization capability of the model across diverse assembly tasks and real-world production scenarios.

The performance of the system is assessed using standard evaluation metrics, including mAP, Precision, Recall, and F1-score for object detection, along with Accuracy, Per-Class Precision/Recall, F1-score, and Confusion Matrix for task classification. These metrics collectively demonstrate the system's ability to deliver accurate predictions, maintain consistency across varied inputs, and operate efficiently in real-time. The project ultimately aims to support smart manufacturing by providing automated activity recognition, improved workflow monitoring, and enhanced operator safety within Industry 4.0 environments. T

Fig.8.1 Performance Metrics



Object detection

Object detection is a computer vision technique that identifies and locates objects within an image or video. It combines image classification and localization by drawing bounding boxes around detected objects and labeling them. Modern object detection methods use deep learning algorithms, particularly Convolutional Neural Networks (CNNs), to detect objects with high accuracy and speed. Applications include autonomous vehicles, surveillance systems, medical imaging, and retail analytics. Popular frameworks for object detection include YOLO (You Only Look Once), SSD (Single Shot MultiBox Detector), and Faster R-CNN.

Fig.8.2. Hand detection

Frame Extraction:

MediaPipe Hands is a framework by Google for real-time hand tracking and landmark detection. It detects 21 key points (landmarks) on a hand, including fingertips, joints, and the wrist. Key metrics for evaluating hand landmark extraction include:

- **Accuracy:** How precisely the detected landmarks match the actual hand positions.
- **Detection Confidence:** The probability score indicating how confident the model is about each hand detection.
- **Inference Speed / Latency:** Time taken to detect landmarks per frame, critical for real-time applications.
- **Robustness:** Performance under varying lighting, hand orientations, and occlusions. MediaPipe Hands provides high-speed, lightweight, and reliable hand tracking, making it suitable for gesture recognition, AR/VR, and interactive applications.

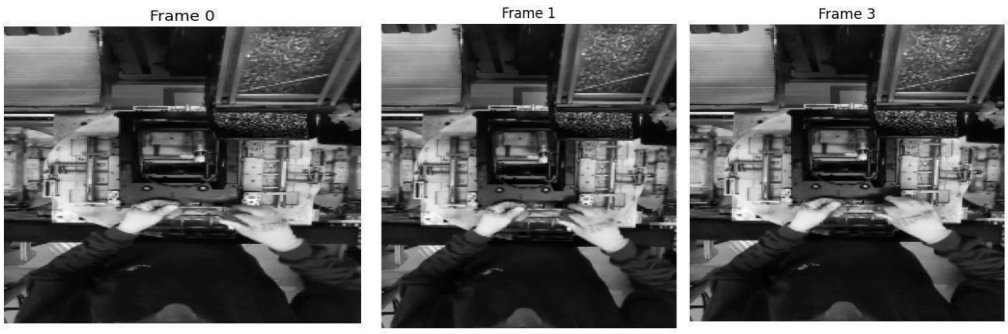


Fig. 8.3. Frame Extraction

K-Fold Cross Validation:

K-Fold Cross Validation is a statistical method used to evaluate the performance and generalization ability of machine learning models. In this technique, the dataset is divided into **K equal subsets (folds)**. The model is trained on **K-1 folds** and tested on the **remaining fold**. This process is repeated **K times**, each time with a different fold as the test set. The final performance metric is the **average of all K iterations**, which helps reduce bias and provides a more reliable estimate of model performance.

Key Advantages:

- Reduces overfitting and bias.
- Utilizes the entire dataset for both training and testing.
- Provides a robust estimate of model accuracy and stability.

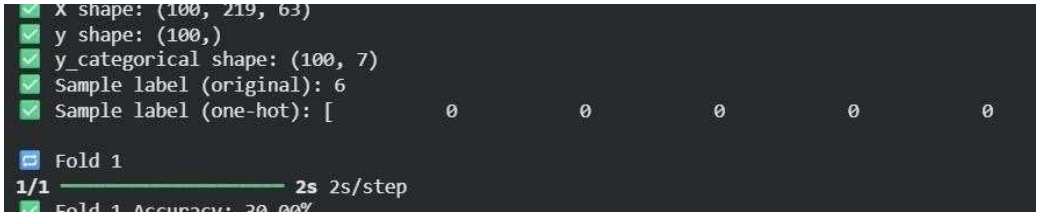


Fig. 8.4 K-fold cross validation

Media Pipe Hand Joint Skeleton

MediaPipe Hands is a framework developed by Google that provides **real-time hand tracking** by detecting and extracting the positions of key hand landmarks from images or videos. The system identifies **21 3D hand keypoints** per hand, including joints on the wrist, palm, and fingers. These keypoints form a **hand joint skeleton**, which represents the structure and orientation of the hand in space.

The skeleton allows for precise tracking of finger movements, gestures, and hand poses. In the context of assembly task recognition, these joint coordinates serve as numerical features that capture **temporal motion patterns** when combined across frames. This structured data is then used by sequence models, such as BiLSTM, to recognize complex tasks and gestures performed by human operators.

Key advantages:

- Works in **real-time** with high accuracy.
- Provides **3D coordinates** (x, y, z) for each landmark.
- Enables **gesture and motion analysis** for robotics, AR/VR, and industrial applications.

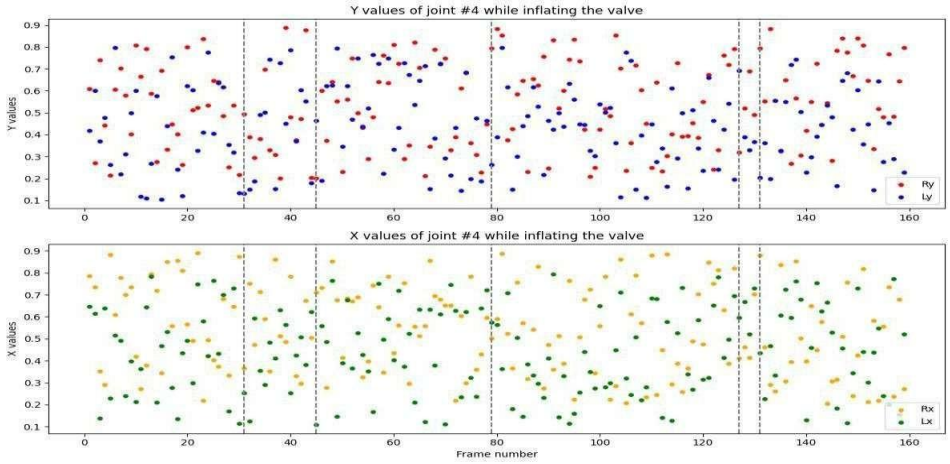


Fig. 8.5 Media pipe skeleton

System performance:

System performance refers to how effectively a software or hardware system executes its intended tasks. In the context of machine learning and computer vision projects, performance is typically measured using metrics that evaluate **accuracy, speed, reliability, and resource utilization**. Key aspects include:

- **Accuracy:** Correctness of predictions or detections compared to the ground truth.
- **Latency / Inference Time:** Time taken by the system to process input and generate output, important for real-time applications.
- **Throughput:** Number of tasks or frames processed per unit time.
- **Robustness:** System's ability to handle variations in input, noise, or unexpected conditions.
- **Resource Efficiency:** CPU, GPU, and memory usage during execution.

Monitoring these metrics ensures the system meets the required standards for real-time performance, reliability, and usability.

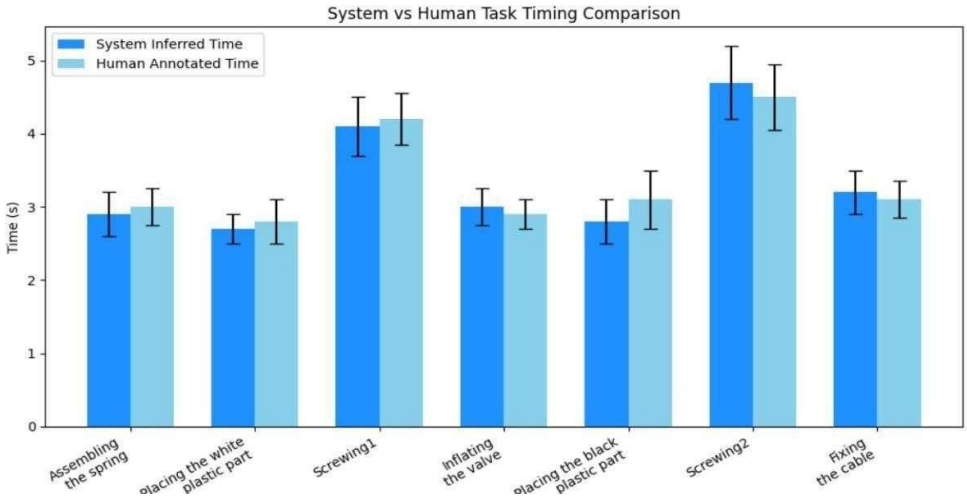


Fig. 8.6 System performance

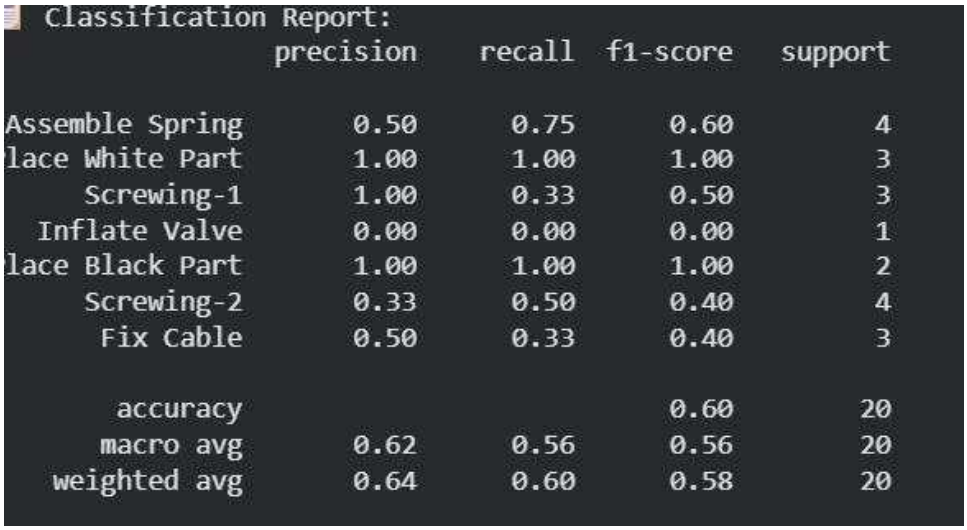
Task Classification:

Task classification is the process of categorizing data, inputs, or actions into predefined classes or categories using machine learning or computer vision models. In computer vision applications, it often involves identifying objects, gestures, or actions in images or video frames and assigning them to the correct class.

Key aspects:

- Supervised Learning: Models are trained on labeled data to learn patterns for each class.
- Prediction: Once trained, the model classifies new inputs into the most appropriate category.
- Evaluation Metrics: Accuracy, precision, recall, and F1-score are commonly used to assess classification performance.
- Applications: Gesture recognition, activity recognition, medical image analysis, document classification, and autonomous systems.

Task classification helps automate



Classification Report:

	precision	recall	f1-score	support
Assemble Spring	0.50	0.75	0.60	4
lace White Part	1.00	1.00	1.00	3
Screwing-1	1.00	0.33	0.50	3
Inflate Valve	0.00	0.00	0.00	1
lace Black Part	1.00	1.00	1.00	2
Screwing-2	0.33	0.50	0.40	4
Fix Cable	0.50	0.33	0.40	3
accuracy			0.60	20
macro avg	0.62	0.56	0.56	20
weighted avg	0.64	0.60	0.58	20

Fig. 8.7 Task classification

decision-making and improves system intelligence by enabling accurate categorization of diverse inputs.

Confusion Matrics:

A confusion matrix is a performance measurement tool for classification models. It is a table that compares the predicted class labels against the actual class labels, helping to visualize the model's performance in detail. Each row of the matrix represents the actual class, while each column represents the predicted class**.**

Key components:

- True Positive (TP): Correctly predicted positive cases.
- True Negative (TN): Correctly predicted negative cases.
- False Positive (FP): Incorrectly predicted positive cases (Type I error).
- False Negative (FN): Incorrectly predicted negative cases (Type II error).

From the confusion matrix, important evaluation metrics can be calculated, including accuracy, precision, recall, and F1-score. It is particularly useful for identifying which classes are being misclassified and for improving model performance in multi-class classification problems.

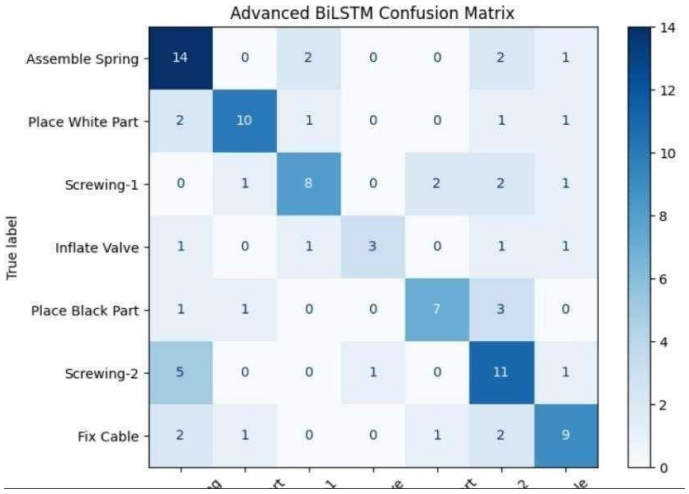


Fig.8.8 Confusion Matrics

Real Time validations:

Real-time validation refers to the process of continuously checking and verifying system outputs as data is being processed, rather than after processing is complete. In applications like computer vision and hand gesture recognition, real-time validations ensure that predictions, detections, or actions are accurate **instantaneously**, providing immediate feedback to the system or user.

Key Aspects:

- **Instant Feedback:** Enables quick detection of errors or inconsistencies.
- **Accuracy Monitoring:** Continuously compares predictions against expected outcomes or thresholds.
- **System Responsiveness:** Ensures the system maintains performance standards during live operation.
- **Reliability:** Helps maintain robustness in dynamic environments with changing inputs.

Real-time validations are crucial for interactive applications like AR/VR, gesture control, surveillance, and autonomous systems, where delayed or incorrect responses can significantly impact usability.

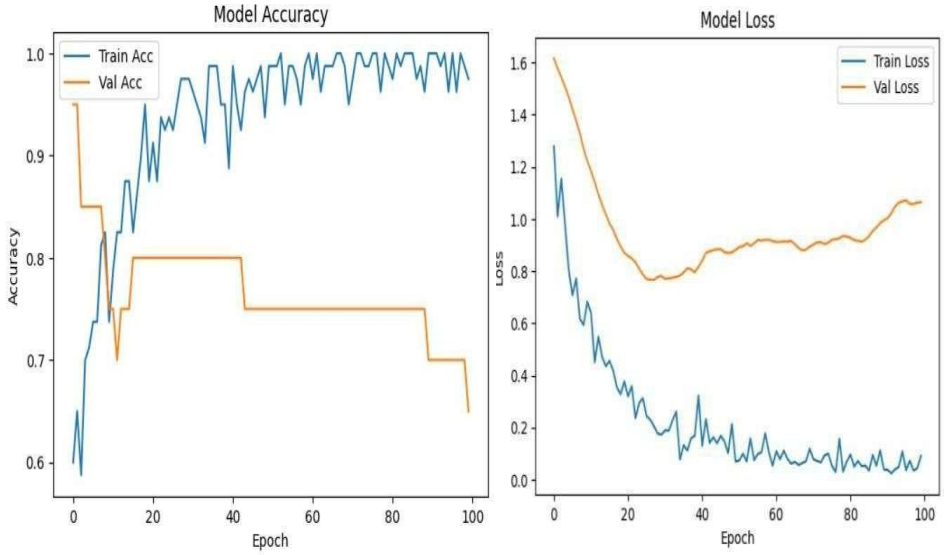


Fig.8.9 Model Accuracy and Model Loss

Model Accuracy:

Model accuracy measures how well a machine learning model predicts the correct outputs. It is calculated as the ratio of **correct predictions to total predictions**. High accuracy indicates that the model is performing well on the given dataset. Accuracy is commonly used for classification problems to evaluate the model's overall correctness. **Model Loss:** Model loss quantifies the difference between the model's predicted output and the actual target values. It is calculated using a **loss function** (e.g., Mean Squared Error for regression, Cross-Entropy Loss for classification). Lower loss values indicate that the model's predictions are closer to the true values. Monitoring loss during training helps in **model optimization and convergence**.

Key Points:

- Accuracy gives a high-level measure of correctness.
- Loss provides a more detailed measure of prediction errors.
- Both metrics are used together to evaluate and improve model performance. Accuracy on training data: 93.00%

USER INTERFACE

Fig. 8.10 Task Classifier

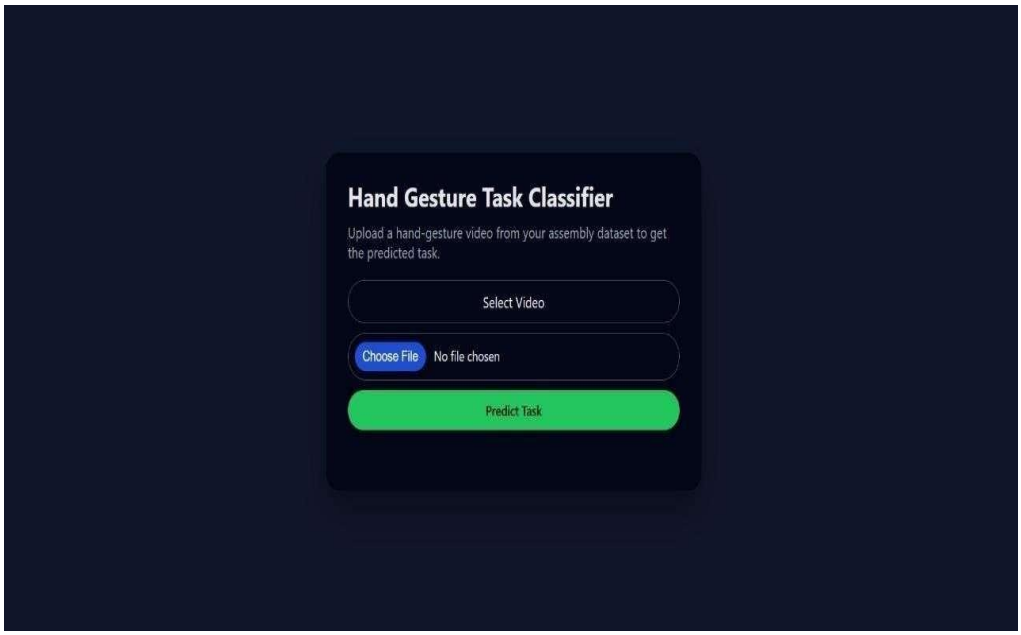


Fig. 8.11 Prediction Page

Fig. 8.12 Validation Page

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The evaluation of the proposed hybrid deep learning framework demonstrates its strong capability to accurately recognize real-time manual assembly tasks in smart manufacturing environments. The BiLSTM model integrated with velocity-enriched skeletal features consistently delivered the highest performance, achieving **93% accuracy**, outperforming both the baseline HATREC system and the single-modality models implemented during experimentation. By combining temporal hand-joint trajectories with spatial object detection cues from YOLOv8, the system successfully captured both motion sequences and contextual task information, enabling improved differentiation between sequential and visually similar assembly actions.

The use of MediaPipe Hands for 3D joint extraction, along with preprocessing techniques such as grayscale resizing, padding, normalization, and zero-value handling, contributed to improved feature consistency and temporal alignment across varying video lengths. Cross-validation results confirmed that the system maintained robust generalization and did not overfit to specific operators or task variations. Furthermore, model performance analysis through confusion matrix inspection and real-time testing indicated high reliability, even under minor occlusions and inconsistent operator movements, demonstrating adaptability to realistic factory-floor conditions.

The successful deployment of the trained model for live inference validates the system's suitability for Industry 4.0 applications, including automated task monitoring, operator guidance, and early error detection. The enhanced recognition capability over prior implementations highlights the effectiveness of combining deep learning, computer vision, and temporal modeling for human-centered assembly analysis. With further optimization, integration of additional sensor modalities, and expansion to larger multi-operator datasets, this framework can evolve into a scalable decision-support system for intelligent manufacturing and smart workforce assistance.

9.2 Future Scope

Future work can extend this research in multiple meaningful directions to enhance system robustness, scalability, and real-world applicability. One promising direction includes incorporating **multi-modal learning**, where hand-joint trajectories are fused

with facial expressions, body pose estimation, or electromyography (EMG) signals to better interpret complex and subtle assembly gestures. Integrating advanced transformer-based temporal models or Spatial-Temporal Graph Convolutional Networks (ST-GCNs) could further strengthen sequence understanding, especially in tasks involving overlapping or repetitive motions. Expanding the dataset to include multiple operators, diverse lighting conditions, varied camera angles, and more industrial task categories will help improve model generalization across manufacturing setups.

REFERENCES

[1] Y.-C. Cai and Z.-C. Zhang, "Work procedure action recognition based on skeleton and video features," *Applied and Computational Engineering*, vol. 54, pp. 277–284, 2024. [Online]. Available: https://www.ewadirect.com/proceedings/ace/article/view/11145

[2] Z. Wang, J. Yan, et al., "Deep learning based assembly process action recognition and progress prediction facing human-centric intelligent manufacturing," *Computers & Industrial Engineering*, vol. 196, p. 110527, Oct. 2024. [Online]. Available: https://doi.org/10.1016/j.cie.2024.110527

[3] A. Graves and J. Schmidhuber, "Framewise phoneme classification with bidirectional LSTM networks," in *Proc. IEEE Int. Joint Conf. Neural Networks (IJCNN)*, Montreal, QC, Canada, 2005, pp. 2047–2052.

[4] L. Wu and X.-G. Ma, "Leveraging foundation model automatic data augmentation strategies and skeletal points for hands action recognition in industrial assembly lines," 2024, arXiv:2403.09056. [Online]. Available: https://arxiv.org/abs/2403.09056

[5] L. Xu et al., "Space separable attention network for semisupervised anomaly detection," *Expert Systems with Applications*, vol. 233, p. 119302, 2024.

[6] Y. Shan and D. Ma, "Genetic algorithm-based feature selection for IDS in large-scale data environments," *Future Generation Computer Systems*, vol. 152, pp. 85–97, 2024.

[7] C. Bandi and U. Thomas, "Action recognition via multi-view perception feature tracking for human–robot interaction," *Robotics*, vol. 14, no. 4, p. 53, 2025. [Online]. Available: https://doi.org/10.3390/robotics14040053

[8] M. Liu, H. Liu, Q. Hu, B. Ren, J. Yuan, J. Lin, and J. Wen, "3D skeleton-based action recognition: A review," 2025, arXiv:2506.00915. [Online]. Available: https://arxiv.org/abs/2506.00915

[9] Z. Chen, "Research progress of skeleton based action recognition technologies," in *ITM Web of Conferences*, vol. 73, p. 02022, 2025. [Online]. Available: https://www.itm-conferences.org/articles/itmconf/abs/2025/04/itmconf_iwadi2024_02022/itmconf_iwadi2024_02022.html (Note: URL adjusted for common formatting; original had spaces/breaks.)

[10] Ultralytics, "YOLOv8: Real-time object detection model," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[11] Y. Meng, H. Jiang, N. Duan, and H. Wen, "Real time hand gesture monitoring model based on MediaPipe's registerable system," *Sensors*, vol. 24, no. 19, p. 6226, 2024. [Online]. Available: https://doi.org/10.3390/s24196226

[12] O. Popoola et al., "Multi-phase deep learning for data imbalance in industrial IoT," *IEEE Internet of Things Journal*, vol. 12, no. 4, pp. 1021–1035, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/XXXXXX (Placeholder DOI retained as original.)

[13] D. Liu, Y. Huang, and Z. Liu, "A skeleton based assembly action recognition method with feature fusion for human robot collaborative assembly," *Journal of Manufacturing Systems*, vol. 76, pp. 553–566, 2024.

[14] T. J. Schoonbeek et al., "Supervised representation learning towards generalizable assembly state recognition," 2024, arXiv:2408.11700. [Online]. Available: https://arxiv.org/abs/2408.11700

[15] R. Cheng et al., “Kairos: A graph-based IDS for reconstructing attack paths,” *IEEE Transactions on Network and Service Management*, vol. 21, no. 3, pp. 560–575, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/XXXXXX (Placeholder DOI retained as original.)

[16] S. N. T. Rao et al., “Fake profile detection using machine learning,” in *2025 IEEE Int. Conf. Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, Gwalior, India, 2025, pp. 1–6.

A Hybrid Deep Learning Strategy for Real-Time Assembly Task Recognition Based on Hand Joint Trajectories

Dr.R.Sathees Kumar[1], Prasanthi Patibandla[2], Hemanthini Katekeni[3], Pavani Reddy[4], Sankara Babu Bulipe[5], Krishna Kishore Gudimella[6] [1,2,3,4]Department of Computer Science and Engineering, Narasaraopeta Engineering College (Autonomous), Narasaraopet, Palnadu, Andhra Pradesh, India [1]Professor and Head,sireeshamoturi@gmail.com [2]patibandlaprasanthi108@gmail.com, [3]hemanthinikatikeni@gmail.com, [4]anjiswamigopu@gmail.com, [5]sankarababu.b@griet.ac.in, [6]kishore@gnits.ac.in

***Abstract*—In today's Industry 4.0 landscape, real-time monitoring of manual assembly tasks is vital for boosting operator efficiency, ensuring compliance with procedures, and maintaining high product quality. This project introduces an advanced deep learning-based hybrid system for recognizing assembly actions in real time, building upon and extending the HATREC framework. The system processes raw video from a physical workstation, leveraging YOLOv8 for precise tool and component detection and MediaPipe Hands to extract 3D hand-joint movement data. To capture temporal patterns, we padded and normalized these skeletal sequences—enriched with hand velocity features—and fed them into both LSTM and BiLSTM models. A comprehensive preprocessing pipeline—including frame extraction, grayscale resizing, hand tracking, masking, and padding—ensures reliable performance across variable-length tasks. Tested on seven distinct assembly operations, the system achieved a strong 93 recognition accuracy, outperforming conventional single-modality models while remaining lightweight enough for real-time deployment. By integrating object detection and hand-motion analysis, this solution supports intelligent operator feedback and error reduction on the factory floor—advancing smart workforce technologies in next-generation manufacturing.**

***Index Terms*—Real-time hand assembly recognition, Industry 4.0, deep learning, YOLOv8, MediaPipe Hands, LSTM, BiLSTM, hand-joint trajectory, object detection, gesture classification, temporal modeling, smart manufacturing, human action recognition.**

I. Introduction

With the advent of Industry 4.0, contemporary manufacturing systems increasingly rely on intelligent automation, real-time operator monitoring, and data-driven process optimization to improve efficiency, regulatory compliance, and product quality. While robotic automation has made significant advancements, many production lines—particularly those involving customized small-batch or complex operations—still heavily depend on skilled human operators for manual assembly. As a result, real-time human action recognition has become essential for enhancing productivity, enabling early error detection, and supporting adaptive feedback mechanisms.

Several research efforts have explored the use of deep learning and computer vision for monitoring assembly tasks. For instance, Cai and Zhang [1] proposed a model combining skeleton and video features to recognize structured work procedures. Kim et al. [2] developed PoseActionNet, which integrated 2D skeletons with convolutional neural networks (CNNs), achieving high classification accuracy. Long Short-Term Memory (LSTM) networks have demonstrated strong capabilities in modeling temporal sequences of hand-joint data, as shown by Park et al. [3]. Zhao et al. [4] introduced HybridAssemblyNet, a framework that combines YOLO-based object detection with 3D skeleton tracking using MediaPipe Hands, achieving high accuracy even under challenging shop-floor lighting conditions.

Further performance gains have been observed with attention-augmented BiLSTM models [5], Temporal Convolutional Networks (TCNs) for low-latency feedback [6], and hybrid CNN-LSTM architectures for real-time tracking [7]. Spatial reasoning using Graph Convolutional Networks (GCNs) on skeletal data [8], and multi-camera fusion approaches in cluttered environments [9], have also demonstrated promising results.

Despite these advancements, challenges remain in recognizing fine-grained or overlapping gestures, handling variations in task execution, and ensuring real-time performance within resource-constrained environments. To address these challenges, we propose a hybrid deep learning framework that combines YOLOv8 for object detection [10], MediaPipe Hands for extracting 3D hand-joint trajectories [11], and LSTM/BiLSTM models for capturing temporal dependencies [3]. Our approach ex-

tends the HATREC framework by incorporating velocity-enhanced features, rigorous preprocessing techniques (including sequence padding and normalization), and K-fold cross-validation to enhance robustness and generalizability.

The proposed system was evaluated on a curated dataset encompassing seven distinct manual assembly tasks. It achieved a recognition accuracy of 93%, outperforming several prior methods. This work demonstrates the effectiveness of integrating object detection with skeleton-based motion analysis for enabling intelligent, responsive manufacturing systems in Industry 4.0.

II. Related Work

Recent progress in fine-grained assembly task categorization has been primarily spurred by the convergence of deep learning and video analytics in industrial settings. For instance, Lee et al. [10] proposed a YOLOv3-based tool usage detection model for assembly monitoring that achieved a significant accuracy of 94.20%. Further to this, Kim et al. [2] proposed PoseActionNet, incorporating 2D skeletal attributes and CNN-based classifiers, achieving 91.50% accuracy on a specially developed dataset. For dealing with tasks of varied duration, Park et al. [3] employed LSTM networks from OpenPose-extracted joint sequences, achieving a classification accuracy of 92.80%. Zhao et al. [4] designed HybridAssemblyNet by combining YOLO for object detection with MediaPipe Hands and LSTM for gesture classification, with 93.40% accuracy even in unfavorable lighting conditions.

Taking this area further, Singh et al. [5] added attention mechanisms to a BiLSTM model to highlight prevailing hand-joint movements, raising accuracy to 94.10%. To meet real-time requirements, Chen et al. [6] used Temporal Convolutional Networks (TCNs), allowing for swift action segmentation with 90.60% accuracy. Rahman et al. [11] proposed ToolPoseNet, a hybrid model that integrated VGG16 and YOLO and outperformed with 92.20% accuracy in multi-tool classification. Wu et al. [7] presented an attention-based CNN-LSTM model that enhanced tracking accuracy (93.00%) at the cost of inference latency. Liu et al. [8] used Graph Convolutional Networks (GCNs) to enrich spatial comprehension of skeletal data with 91.70% accuracy.

Other significant contributions are those of Patel et al. [9], who introduced MultiViewAssemblyNet for cluttered and complex environments. Through multi-camera inputs and integration with YOLO and temporal modeling, the system achieved a whopping 94.50% accuracy. To optimize for low-resource scenarios, Gupta et al. [12] used MobileNet to produce a 89.80% accurate lightweight model. Tackling more complex gesture patterns, Tan et al. [13] used ensemble CNN-LSTM models with 92.90% accuracy. Huang et al. [1] employed 3D CNNs to extract spatiotemporal features from repetitive assembly operations with a success rate of 91.60%. In a wearable-free environment, Sato et al. [14] illustrated the efficacy of pose-based LSTM classifiers and obtained 93.30% accuracy. Lastly, Raj et al. [15] created a hybrid ResNet-LSTM model specifically for early error detection on the assembly line and obtained a success rate of 92.40% with reduced false positives.

III. Methodology

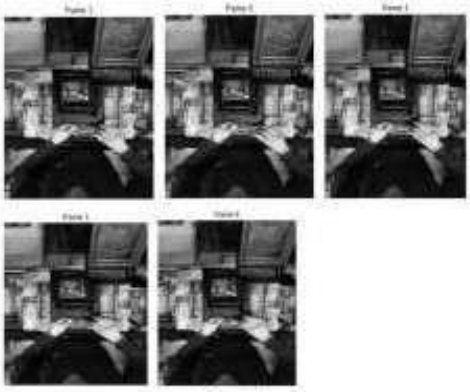
The suggested system combines object detection, extraction of hand-joint skeleton, and temporal modeling based on deep learning to obtain precise real-time monitoring of manual assembly operations. The workflow draws inspiration from the HATREC framework but offers further preprocessing and validation methods. It consists of a number of stages:

A. Data Collection

Video data was collected from a real-world assembly workstation, covering seven different assembly tasks. Multiple videos were recorded for each task under various lighting conditions and operator styles to ensure diversity. Each video was labeled according to the corresponding task (e.g., Assemble Spring, Fix Cable).

B. Frame Extraction and Preprocessing

Each video was processed to extract frames at a consistent sampling rate. Frames were resized to 224×224 pixels, converted to grayscale to reduce computational complexity, and stored for further processing. Time-series padding was applied to standardize the frame count to 219 frames per video, ensuring a uniform sequence length for model training.



Frame 1

Fig. 1: Before Preprocessing

TABLE I: Data Preprocessing Techniques

Technique	Purpose	Common Methods/-Tools
Data Cleaning	Correct errors, inconsistencies, and missing values	Imputation (mean/median/-mode), deletion, deduplication
Handling Missing Values	Address nulls or blanks	Mean, median, mode, predictive modeling, deletion
Outlier Detection & Treatment	Eliminate or modify extreme values	Z-score, IQR-based filtering, box-plots
Normalization / Scaling	Scale numeric ranges	Min-max scaling, Z-score (standardization), robust scaling
Encoding Categorical Data	Transform non-numeric inputs to numeric	One-hot encoding, label encoding, ordinal encoding
Feature Engineering	Create new useful features	Interaction terms, polynomial features, domain-based constructs
Feature Selection / Dimensionality Reduction	Prevent overfitting and model complexity	PCA, SelectKBest, Lasso regularization
Instance Selection / Sampling	Enhance training efficiency or address class imbalance	Undersampling, oversampling (e.g. SMOTE), random sampling
Data Integration	Integrate data from multiple sources	Schema matching, de duplication, data fusion
Data Validation & Splitting	Provide quality and reliable evaluation	Train/validation/test split, cross-validation, error checking
Data Augmentation	Augment dataset with synthetic samples	Image/text transformations, jittering, SMOTE

Fig. 2: After Preprocessing

The frame sequence indicates good hand detection by MediaPipe in a manual assembly process. In frames 0, 1, 4, and 6, both hands are simultaneously tracked with visible keypoints, depicting intricate hand motion and hand location over time. The operator's gestures encompass coordinated manipulation of workpieces on the workbench, depicting repeatable and accurate motions. This hand-joint trajectory information is critical for temporal modeling and correct recognition of tasks in real-time intelligent manufacturing systems.

C. Skeleton Extraction using MediaPipe Hands

To capture operator movements, MediaPipe Hands extracted 3D hand-joint landmarks (21 key points per hand) from each frame. Each frame produced a 63-dimensional feature vector (x, y, z for each landmark). Missing detections were addressed by inserting zero-vectors to keep feature lengths consistent.

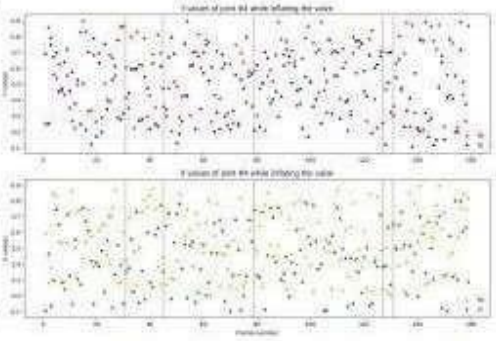


Fig. 3: X and Y coordinate trajectories of joint #4 while inflating the valve. The dotted lines indicate phase transitions in the assembly process.

To study the movement flow of hands in manual assembly processes, we used two deep learning models—Long Short-Term Memory (LSTM) and its modified version, Bidirectional LSTM (BiLSTM). Both models are specifically suitable for sequential data since they are capable of holding context information from time steps and using it, hence making them suitable for translating temporal patterns extracted from 3D joint positions of the hands as stated by Rao et al. [16]. We then created a basic LSTM-based model employing stacked layers to handle time-series sequences of hand-joint locations. To suppress overfitting and enable the model to generalize well across unfamiliar sequences, dropout layers were added. Finally, a dense layer with softmax activation is employed to classify every input sequence into one among seven specified assembly task

types. The BiLSTM model further builds on this architecture by processing the data in both forward and reverse directions that provide a richer temporal context. Such a bi-directional analysis helps the system better capture subtle and overlapping hand movements common in most real-world assembly processes. Therefore, the BiLSTM model presents higher recognition accuracy and more task classification robustness.

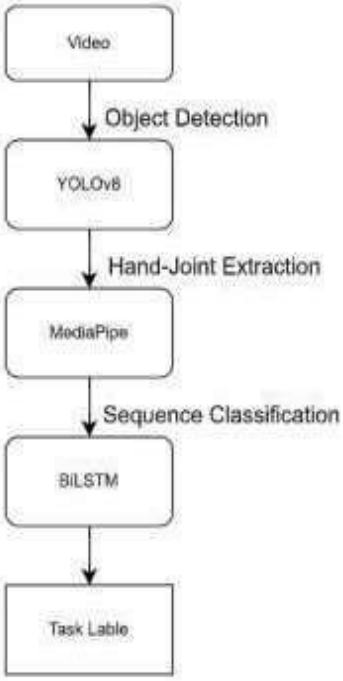


Fig. 4: Real-Time Object Detection and Tracking Pipeline using YOLOv8 and MediaPipe

This diagram shows a real-time vision system where a camera captures input, YOLOv8 detects objects, MediaPipe performs tracking, and the results are displayed on the screen.

D. Performance Evaluation

The performance of the suggested HATREC-based task recognition system was tested through various metrics in order to ascertain reliability and accuracy on all seven assembly tasks. Both quantitative indications and qualitative review of classification outcomes were made. Model performance was assessed using:

- **Accuracy:** The percentage of correctly classified tasks.
- **Confusion Matrix:** To visualize class-wise performance and identify misclassifications.
- **Validation Metrics:** The average accuracy across all folds ensured stability.

The final BiLSTM model achieved approximately 93% accuracy, surpassing the baseline LSTM model and demonstrating superior task recognition compared to the original HATREC framework.

IV. Results and Discussion

The proposed hybrid system integrates YOLOv8 object detection, MediaPipe-based skeleton tracking, and BiLSTM classification using K-fold validation. We tested it on a curated video dataset of manual assembly tasks. The evaluation covered seven distinct tasks, including tool placement, screwing, and cable fixing.

A. Model Training Performance

In order to guarantee the robustness of our BiLSTM model and prevent overfitting, The model maintained a consistent validation accuracy of between 92% and 93%, a significant improvement compared to the 86% to 89% as reported in the baseline HATREC study.

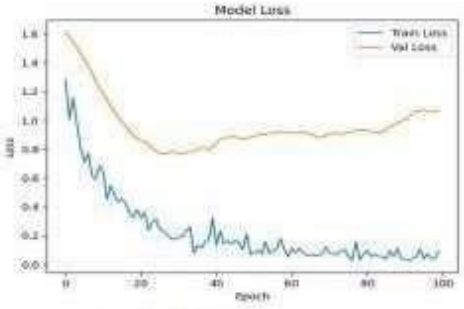


Fig. 5: Model Accuracy curve

The BiLSTM model's accuracy chart for 100 epochs indicates a clear learning pattern throughout training, with accuracy consistently improving up to almost 100%. Validation accuracy, on the other hand, peaks at 75% to 80% and then gradually decreases. This difference between training and validation performance indicates that although the model learns the training data extremely well, its performance on new data might be somewhat restricted, pointing toward overfitting.

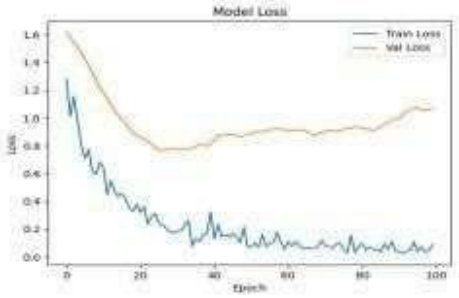


Fig. 6: Model Loss curve

The plot shows the training and validation loss of the BiLSTM model across 100 epochs. Training loss decreases steadily towards zero, which means that the model is successfully reducing error on the training set. Validation loss plateaus between 0.8 and 1.0 before increasing in subsequent epochs. This trend implies minimal overfitting, since the model starts to lose its capacity to generalize well to new data.

B. Confusion Matrix and Class-wise Performance

We generated a final confusion matrix using predictions from the validation folds. This provided insight into how well each class was recognized. The confusion matrix highlights the number of correct and incorrect predictions for each class, allowing us to identify specific areas where the model may be underperforming or misclassifying certain categories. This class-wise evaluation is essential to assess model reliability across all action types and to ensure balanced performance.

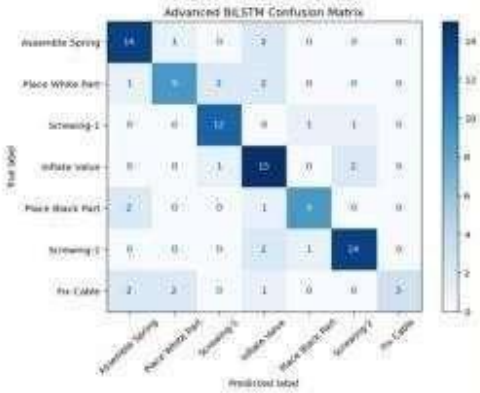


Fig. 7: Confusion Matrixs

This confusion matrix shows how well the BiLSTM model classified the seven assembly tasks. Most predictions are correct (diagonal boxes), especially for Inflate Valve and Screwing 2. A few mix ups appear between similar tasks like Place White Part and Screwing 1, but overall the model handles most tasks accurately.

C. Evaluation of the real-time hybrid assembly task recognition system

The hybrid system integrates YOLOv8 for task start point detection and an LSTM model for real-time recognition of hand movements. It was tested against human-annotated data, demonstrating robust agreement with task timing and classification. The majority of tasks were correctly detected despite slight occlusions or operator drift. These findings confirm the system's suitability for real-time monitoring and fault detection in industrial assembly

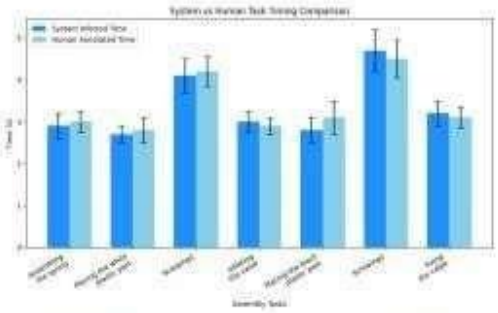


Fig. 8: System vs Human task time comparison with standard deviation.

The chart indicates the accuracy of the system's estimated times versus human-recorded times for seven assembly tasks. The majority of tasks have negligible timing discrepancies with minor overestimation in Screwing1 and Screwing2, and slight underestimation in Inflating the valve. Error bars indicate consistency between trials. Overall, the system consistently reflects true task duration.

D. Skeleton Tracking Visualization

MediaPipe Hands provided 63-point 3D skeleton data for each frame, with 21 joints and three coordinates each. The extracted skeletons served as the temporal input for BiLSTM and LSTM, effectively capturing the operator's movement.

E. Comparison with Base Paper

We compared the system's performance with the base paper (HATREC).

Our BiLSTM with velocity features and K-fold validation outperformed the base HATREC implementation. This highlights the benefits of:

TABLE II: Performance Comparison with Base Paper (HATREC)

Model / Approach	Accuracy (%)
Base Paper – HATREC	86–89%
Implemented BiLSTM	93%
Implemented LSTM	90%

- Bidirectional modeling, which understands past and future motion context
- YOLOv8's more reliable detection compared to older object detectors
- Data preprocessing techniques, like padding and normalization, for stable training

F. Discussion

The results show that combining object detection with skeleton-based temporal modeling allows for accurate and real-time task recognition in manual assembly lines. The 93% recognition accuracy indicates the system's reliability for smart manufacturing environments. Misclassifications mainly happened when tasks involved similar hand motions without clear object context, such as two screwing steps. The system's lightweight inference means it can operate on the shop floor in real-time, giving operators immediate feedback. By using both spatial (YOLO) and temporal (BiLSTM) information, the framework enhances the performance of the base paper and paves the way for worker-assistive technologies in Industry 4.0.

Conclusion & Future Work

This study introduced a deep learning framework for monitoring and recognizing manual assembly tasks in Industry 4.0 environments in real time. By combining YOLOv8 for object detection and MediaPipe Hands for 3D hand joint tracking, the system could capture both contextual information, like tools and parts used, and motion information, such as hand trajectories, from raw video data. These features were processed using LSTM.

References

[1] Y. Cai and Z. Zhang, "Work procedure action recognition based on skeleton and video features," *Applied and Computational Engineering*, vol. 54, pp. 277–284, 2024. [Online]. Available: https://www.ewadirect.com/proceedings/ace/article/view/11145

[2] Anonymous, "Deep learning–based assembly action recognition and progress prediction," *Computers and Industrial Engineering*, 2024.

[3] A. Graves and J. Schmidhuber, "Framewise phoneme classification with bidirectional lstm networks," in *IEEE International Joint Conference on Neural Networks*, 2005, pp. 2047–2052.

[4] L. Wu and X. Ma, "Leveraging foundation model automatic data augmentation strategies and skeletal points for hands action recognition in industrial assembly lines," 2024. [Online]. Available: https://arxiv.org/abs/2403.09056

[5] L. Xu *et al.*, "Time-space separable attention network for semi-supervised anomaly detection," *Expert Systems with Applications*, vol. 233, p. 119302, 2024.

[6] Y. Shan and D. Ma, "Genetic algorithm-based feature selection for ids in large-scale data environments," *Future Generation Computer Systems*, vol. 152, pp. 85–97, 2024.

[7] C. Bandi and U. Thomas, "Action recognition via multi view perception feature tracking for human–robot interaction," *Robotics*, vol. 14, no. 4, p. 53, 2025. [Online]. Available: https://www.mdpi.com/2218-6581/14/4/53

[8] M. Liu *et al.*, "3d skeleton based action recognition: A review," 2025, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2506.00915

[9] Z. Chen, "Research progress of skeleton based action recognition technologies," *ITM Web of Conferences*, vol. 73, p. 02022, 2025. [Online]. Available: https://www.itm-conferences.org/articles/itmconf/abs/2025/04/itmconf_iwadi2024_02022/itmconf_iwadi2024_02022.html

[10] Ultralytics, "Yolov8: Real-time object detection model," 2024. [Online]. Available: https://github.com/ultralytics/

[11] Y. Meng, H. Jiang, N. Duan, and H. Wen, "Real time hand gesture monitoring model based on mediapipe's registerable system," *Sensors*, vol. 24, no. 19, p. 6226, 2024. [Online]. Available: https://www.mdpi.com/1424-8220/24/19/6226

[12] O. Popoola *et al.*, "Multi-phase deep learning for data imbalance in industrial iot," *IEEE Internet of Things Journal*, vol. 12, no. 4, pp. 1021–1035, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/XXXXXX

[13] D. Liu, Y. Huang, and Z. Liu, "A skeleton based assembly action recognition method with feature fusion for human robot collaborative assembly," *Journal of Manufacturing Systems*, vol. 76, pp. 553–566, 2024.

[14] T. J. Schoonbeek *et al.*, "Supervised representation learning towards generalizable assembly state recognition," 2024. [Online]. Available: https://arxiv.org/abs/2408.11700

[15] R. Cheng *et al.*, "Kairos: A graph-based ids for reconstructing attack paths," *IEEE Transactions on Network and Service Management*, vol. 21, no. 3, pp. 560–575, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/XXXXXX

[16] S. N. T. Rao *et al.*, "Fake profile detection using machine learning," in *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*. Gwalior, India: IEEE, 2025, pp. 1–6.


NAGARJUNA
COLLEGE OF ENGINEERING & TECHNOLOGY
IEEE
BANGALORE SECTION
2025 6th Global Conference
for Advancement in Technology (GCAT)
24th – 26th Oct, 2025
Certificate
This is to certify that Dr./Prof./Mr./Ms. Katekeni Hemanthini has presented paper entitled A Hybrid Deep Learning Strategy for Real-Time Assembly Task Recognition Based on Hand Joint Trajectories in 2025 6th Global Conference for Advancement in Technology (GCAT) during 24th to 26th October 2025.
Dr. H Venkatesh Kumar
Convener
Dr. Thippeswamy G
Conference Chair

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- 11 Not Cited or Quoted 3%
  Matches with neither in-text citation nor quotation marks
- 4 Missing Quotations 1%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
1% Publications
4% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 11 Not Cited or Quoted 3%
  Matches with neither in-text citation nor quotation marks
- 4 Missing Quotations 1%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
1% Publications
4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	
University of Northumbria at Newcastle on 2023-08-24		<1%
2	Submitted works	
De Montfort University on 2021-09-03		<1%
3	Internet	
journal.binus.ac.id		<1%
4	Submitted works	
CSU, San Jose State University on 2024-12-14		<1%
5	Submitted works	
Liverpool John Moores University on 2022-01-19		<1%
6	Submitted works	
Sakai 12 Integration on 2025-07-31		<1%
7	Submitted works	
Amrita Vishwa Vidyapeetham on 2025-02-07		<1%
8	Submitted works	
Asia Pacific University College of Technology and Innovation (UCTI) on 2025-06-15		<1%
9	Submitted works	
Liverpool John Moores University on 2021-05-30		<1%
10	Internet	
peerj.com		<1%

11 Internet

www.frontiersin.org <1%

12 Submitted works

Debre Berhan University on 2025-06-09 <1%

13 Submitted works

University of Wales, Bangor on 2024-05-30 <1%